Assignment 4: MIPS Game Project

Tic-Tac-Toe, Blackjack and Hangman in MIPS assembly

Dylan Flores, SDSU, Fall 2026, October 18, 2026

How to run the games

Each game is one file: tictactoe.asm, blackjack.asm and hangman.asm. They all use the same Bitmap Display settings. All of the code, the Java versions and the display images are also on my GitHub at github.com/dflo042/cs in the mips-game-project folder.

- Open the file in MARS 4.5 and assemble it (F3).
- Tools, then Bitmap Display. Unit Width 4, Unit Height 4, Display Width 256, Display Height 256, Base address 0x10008000 ($gp). Click Connect to MIPS.
- Run (F5). Prompts show up in the Run I/O box at the bottom and the game is drawn in the Bitmap Display window. The board or cards are also printed as text in the console.
- Type an answer and press Enter. Bad input gives an error message and asks again. N at the play again prompt quits.

Java versions: javac TicTacToe.java Blackjack.java Hangman.java, then java TicTacToe, java Blackjack or java Hangman. They take the same input and print a text version of the display after each move.

Controls: Tic-Tac-Toe wants a cell number 1 to 9. Blackjack wants H or S (either case). Hangman wants one letter. All three ask Y or N to play again. Anything else is rejected with a message and nothing changes.

How the display and input work

I wrote one small set of graphics and input procedures and put the same copy in each file so every file assembles on its own. The display is 64 by 64 units, one word per unit holding a color as 0x00RRGGBB, 4096 words starting at 0x10008000. A pixel's address is 0x10008000 + (y * 64 + x) * 4. draw_pixel does that with two shifts and an add and checks that x and y are inside 0 to 63 first so a bad coordinate can never overwrite anything else.

- fill_screen(color) clears the screen. draw_rect(x, y, w, h, color) is a filled rectangle. It has five arguments so the color is passed on the stack and the procedure reads it with lw $t9, 0($sp) before saving its own registers.
- draw_glyph(x, y, char, color) draws one character from a 3 by 5 font I made, stored as 5 bytes per character with bit 2 as the left pixel. It has 0-9, A-Z, space, _, ?, -, : and !. draw_text draws a string 4 pixels per character, so 15 fit across.
- read_line uses syscall 8 to read a whole line and strips the newline. parse_int (Tic-Tac-Toe) turns the text into a number and reports whether it really was one. I did this instead of syscall 5 because syscall 5 throws an error in MARS if you type letters. to_upper lets lower case letters work.

Conventions: $a0-$a3 are arguments, $v0/$v1 return values. $t registers are temporary and never expected to survive a jal (loops that only call draw_pixel keep a counter in $t8 since draw_pixel only uses $t0 and $t1). Every procedure that uses $s registers pushes them and $ra on the stack first and pops them at the end. Leaf procedures do not touch the stack. Game state is in .data as byte arrays and

word counters. .eqv is used for colors and sizes and a macro for the print syscalls.

Game 1: Tic-Tac-Toe

Two players take turns typing a cell number. The board is a 9 byte array (0 empty, 1 X, 2 O) and the eight winning lines are a 24 byte table of cell indexes, so check_win is one loop over that table. After each move the program redraws that cell, runs check_win for the current player, checks move_count for a draw at 9, and switches players with $s0 = 3 - $s0. Empty cells show a dim number so you know what to type, X is two thick diagonals, O is a ring, and when someone wins highlight_line redraws the three winning marks in green and the strip at the bottom says X WINS!, O WINS! or DRAW.

Procedures: main (game loop, prompt and validation, play again), reset_game, check_win(player), highlight_line(line, player), print_console_board and print_cell, show_turn_status and show_status, draw_board_screen, cell_origin(index) which uses div/mflo/mfhi, draw_mark, draw_mark_color, draw_x_mark, draw_o_mark, plus read_line, parse_int and the graphics procedures.



T1: X wins down the middle column



T2: O wins on the diagonal



T3: full board, draw

Tests

- **T1** X wins with the middle column. Typed: 5 1 9 3 2 7 4 6 8, N. Passed.
- **T2** O wins on the anti-diagonal. Typed: 1 3 2 5 4 7, N. Passed.
- **T3** Full board with no winner (draw). Typed: 1 2 3 5 4 6 8 7 9, N. Passed.
- **T4** (edge case) Invalid moves: text, empty line, 0, 10, spaces, occupied cell. Typed: abc, (empty), 0, 10, ' 5 ', 5 (taken), 1, 9, 3, 2, 7, 4, 6, 8, N. Passed.
- **T5** (edge case) Play-again validation and board reset for a second game. Typed: 1 4 2 5 3, maybe, y, 1 2 3 5 4 6 8 7 9, n. Passed.

T4 log, bad input then a taken cell (what I typed is after each prompt)

```
Player X, choose a cell (1-9): abc
  ! That is not a whole number. Enter a digit from 1 to 9.
Player X, choose a cell (1-9):
  ! That is not a whole number. Enter a digit from 1 to 9.
Player X, choose a cell (1-9): 0
  ! Out of range. Enter a digit from 1 to 9.
Player X, choose a cell (1-9): 10
  ! Out of range. Enter a digit from 1 to 9.
Player X, choose a cell (1-9):  5

 1 | 2 | 3
---+---+---
 4 | X | 6
---+---+---
   ...
```

T3 log, end of the draw game

```
Player X, choose a cell (1-9): 9

 X | O | X
---+---+---
 X | O | O
---+---+---
 O | X | X


*** It's a draw. ***
Play again? (Y/N): N
Thanks for playing!
```

Game 2: Blackjack

You play against a dealer that hits on 16 or less and stands on 17 or more. The deck is a 52 byte array holding 0 to 51, where rank is card mod 13 and suit is card / 13. Every round refills and shuffles it with a Fisher-Yates loop using syscall 42, seeded once from the clock (syscalls 30 and 40). deal_card appends the next card to a hand array and bumps its counter. hand_value counts aces as 11 and then subtracts 10 per ace while the total is over 21, so A+6 is 17 and A+6+9 is 16. A natural blackjack is checked first, 21 stands automatically, a bust ends the round, otherwise the dealer flips the hidden card and draws to 17. Wins, losses and pushes are counted across rounds. Cards on the display are 9 by 12 with the rank and a suit letter, red for hearts and diamonds, and the dealer's second card is a blue card back until you stand.

Procedures: main, shuffle_deck, deal_card(hand, count pointer), card_value, hand_value, to_upper, print_card, print_hand, print_player_line, print_table_hidden, print_dealer_full, redraw_table(hide dealer), draw_hand, draw_card, rank_char, draw_scores, draw_record, draw_number and int_to_str (pushes digits on the stack to reverse them).



My turn, dealer card hidden



Round over, 19 vs 18, I won



After the 40 round run

Tests

- **B1** Stand immediately; dealer plays out its hand. Typed: S, N. Passed.
- **B2** Basic strategy: hit under 17, stand at 17+. Typed: H while total < 17, then S; N. Passed.
- **B3** Three rounds in a row; win/loss/push counters accumulate. Typed: basic strategy, Y, Y, N. Passed.
- **B4** (edge case) Invalid commands (7, empty, ?) are rejected without dealing. Typed: 7, (empty), ?, S, N. Passed.
- **B5** (edge case) Hit until bust (or 21), then invalid play-again answer. Typed: H repeatedly, maybe, N. Passed.

The cards are random so every run is different. I also ran 40 rounds with a script that hits under 17 (B6) and a checker recomputed every printed hand total from the card names and checked the dealer's 17 rule and the announced result. 113 totals over 40 rounds, no mistakes, final record 18 won 19 lost 3 pushed.

B3 log, one round of the basic strategy run

```
---------------- NEW ROUND ----------------
Dealer shows : AC  [??]
Your hand    : 8D 9H    (total 17)
(H)it or (S)tand? S
Dealer reveals: AC 6H
Dealer hand  : AC 6H    (total 17)

*** Push - it's a tie. ***
Record: 0 won, 0 lost, 1 pushed.
Play another round? (Y/N): Y

---------------- NEW ROUND ----------------
   ...
```

B4 log, bad commands are rejected

```
---------------- NEW ROUND ----------------
Dealer shows : JS  [??]
Your hand    : 8H 6D    (total 14)
(H)it or (S)tand? 7
  ! Please type H to hit or S to stand.
(H)it or (S)tand?
  ! Please type H to hit or S to stand.
(H)it or (S)tand? ?
  ! Please type H to hit or S to stand.
(H)it or (S)tand? S
Dealer reveals: JS QS
Dealer hand  : JS QS    (total 20)

*** Dealer wins. ***
   ...
```

Game 3: Hangman (my custom game)

I picked Hangman because it uses the requirements differently from the other two: a table of pointers to strings for the word list, a 26 entry array indexed by letter minus 'A' for the letters tried, character checking on input, and a drawing that grows one piece at a time. The word is picked with syscall 42 from twelve computer words (COMPUTER, ASSEMBLY, REGISTER, PROCEDURE, MEMORY, BRANCH, STACK, SYSCALL, PIPELINE, COMPILER, HARDWARE, PROGRAM). A guess is accepted only if it is exactly one character, a letter, and not tried yet. A repeat does not cost a life. count_in_word gives how many times the letter appears, word_solved checks the word against the guessed array, and six misses loses, which reveals the word with the missing letters in red. draw_figure(n) draws the first n parts of the person (head, body, arms, legs) from the wrong guess counter, so the scene is rebuilt from the state every redraw.

Procedures: main, reset_game, count_in_word(letter), word_solved, to_upper, print_state, draw_scene, draw_lives, draw_gallows, draw_figure(parts), draw_diag(x, y, dx), draw_word_row(reveal all), draw_miss_row, draw_number, int_to_str.



H1: solved, no misses



H2: solved after three misses



H3: six misses, word revealed

Tests

- **H1** Solve the word with no misses (word-aware guesser). Typed: letters chosen from the word list; N. Passed.
- **H2** Win after three misses (figure partly drawn). Typed: Z Q J, then word-aware letters; N. Passed.
- **H3** Six misses: the game is lost and the word is revealed. Typed: F J Q V X Z, N. Passed.

- **H4** (edge case) Invalid guesses: two letters, empty, digit, symbol, repeat (no life lost). Typed: ab, (empty), 1, ?, e, E (repeat), F J Q V X Z, N. Passed.
- **H5** (edge case) Lower-case input accepted; play-again validation; new word and 6 lives on replay. Typed: f j q v x z, xyz, Y, F J Q V X Z, n. Passed.

The word is random, so for H1 and H2 a small script picked the letter that appears in the most words still matching the pattern. The losing tests use F, J, Q, V, X and Z because none of the twelve words have those letters.

H4 log, bad guesses and a repeat

```
Guess a letter: ab
  ! Enter exactly one letter.
Guess a letter:
  ! Enter exactly one letter.
Guess a letter: 1
  ! That is not a letter (A-Z only).
Guess a letter: ?
  ! That is not a letter (A-Z only).
Guess a letter: e
  Yes! 'E' appears 1 time(s).
Word : _ _ _ _ _ _ E _    Lives: 6   Missed:
Guess a letter: E
  ! You already tried that letter.
Guess a letter: F
   ...
```

H2 log, end of the game

```
*** You got it! The word was SYSCALL. ***
Score: 1 won, 0 lost.
Play again? (Y/N): N
Thanks for playing!
```

Test summary

Every test ran on the MIPS program in MARS and on the Java program with the same input. 31 of 31 passed. Edge cases covered: letters and blank lines where a number goes, numbers out of range, taken cells, bad commands, multi character and non letter guesses, repeated guesses, bad play again answers, state resetting between games, soft aces, natural blackjack, the dealer's 17 rule, and a bust ending the round before the dealer plays.

Test	Game	MIPS	Java
T1 X wins with the middle column	tictactoe	pass	pass
T2 O wins on the anti-diagonal	tictactoe	pass	pass
T3 Full board with no winner (draw)	tictactoe	pass	pass
T4 Invalid moves: text, empty line, 0, 10, spaces, occupied cell	tictactoe	pass	pass
T5 Play-again validation and board reset for a second game	tictactoe	pass	pass
B1 Stand immediately; dealer plays out its hand	blackjack	pass	pass
B2 Basic strategy: hit under 17, stand at 17+	blackjack	pass	pass
B3 Three rounds in a row; win/loss/push counters accumulate	blackjack	pass	pass
B4 Invalid commands (7, empty, ?) are rejected without dealing	blackjack	pass	pass
B5 Hit until bust (or 21), then invalid play-again answer	blackjack	pass	pass
H1 Solve the word with no misses (word-aware guesser)	hangman	pass	pass
H2 Win after three misses (figure partly drawn)	hangman	pass	pass
H3 Six misses: the game is lost and the word is revealed	hangman	pass	pass

Test	Game	MIPS	Java
H4 Invalid guesses: two letters, empty, digit, symbol, repeat (no life lost)	hangman	pass	pass
H5 Lower-case input accepted; play-again validation; new word and 6 lives on replay	hangman	pass	pass
B6 40 round blackjack check	blackjack	pass	-

Java planning code

I wrote these first to work out the game logic and the display layout. Each static method became a MIPS procedure with the same name, the arrays and counters match the .data section, and the Display class is a model of the Bitmap Display with the same font and drawing routines. The Display class is identical in all three files so it is only listed once, in TicTacToe.java.

TicTacToe.java

```
 1 import java.io.BufferedReader;
 2 import java.io.IOException;
 3 import java.io.InputStreamReader;
 4
 5 /**
 6  * TicTacToe.java -- Java planning / reference version of tictacto
       e.asm
 7  *
 8  * Module 4: MIPS Game Project (Game 1 of 3)
 9  * Author  : Dylan Flores
10  *
11  * This program was written first to plan the MIPS version. Every
       static
12  * method here maps to a procedure with the same name in tictactoe
       .asm, the
13  * board is the same 9-byte array, and the Display class simulates
       the MARS
14  * Bitmap Display (64 x 64 words at 0x10008000) with the same 3x5
       font and
15  * the same drawing routines. The display is printed as text after
       every
16  * move using half-block characters (two pixel rows per text line)
       .
17  *
18  * Run:  javac TicTacToe.java && java TicTacToe
19  */
20 public class TicTacToe {
21
22     // ---- constants (mirror the .eqv values in the .asm file) --
       --
23     static final int COLOR_BG   = 0x202830;
24     static final int COLOR_GRID = 0xE0E0E0;
25     static final int COLOR_X    = 0xFF5050;
26     static final int COLOR_O    = 0x50A0FF;
27     static final int COLOR_WIN  = 0x50FF70;
28     static final int COLOR_HINT = 0x606878;
29     static final int COLOR_TEXT = 0xFFE060;
30     static final int BOARD_X = 8, BOARD_Y = 8, CELL_PITCH = 16, CE
       LL_SIZE = 15;
31     static final int PLAYER_X = 1, PLAYER_O = 2;
32
33     // ---- game state (mirrors the .data section) ----
34     static final int[] board = new int[9];          // 0 empty, 1
       X, 2 O
35     static final int[][] WIN_LINES = {
36         {0, 1, 2}, {3, 4, 5}, {6, 7, 8},             // rows
37         {0, 3, 6}, {1, 4, 7}, {2, 5, 8},             // columns
38         {0, 4, 8}, {2, 4, 6}                         // diagonals
39     };
40     static int moveCount = 0;
41     static final Display display = new Display(COLOR_BG);
42     static final BufferedReader in = new BufferedReader(new InputS
       treamReader(System.in));
43
44     public static void main(String[] args) throws IOException {
45         System.out.print("=== TIC-TAC-TOE (Java reference) ===\n"
46                 + "Two players take turns. Type the number of an e
       mpty cell (1-9).\n\n");
47         boolean playing = true;
48         while (playing) {
49             resetGame();
50             drawBoardScreen();
51             int player = PLAYER_X;
52             while (true) {
53                 printConsoleBoard();
54                 showStatus(player == PLAYER_X ? "X TURN" : "O TURN
       ");
55                 display.render();
56                 int cell = askMove(player);           // validated
        0..8
57                 board[cell] = player;
58                 moveCount++;
59                 drawMark(cell, player, player == PLAYER_X ? COLOR_
       X : COLOR_O);
60                 int line = checkWin(player);
61                 if (line >= 0) {
62                     highlightLine(line, player);
63                     printConsoleBoard();
64                     showStatus(player == PLAYER_X ? "X WINS!" : "O
        WINS!");
65                     display.render();
66                     System.out.println("\n*** Player " + (player =
       = PLAYER_X ? "X" : "O") + " wins! ***");
67                     break;
68                 }
69                 if (moveCount == 9) {
70                     printConsoleBoard();
71                     showStatus("DRAW");
72                     display.render();
73                     System.out.println("\n*** It's a draw. ***");
74                     break;
75                 }
76                 player = 3 - player;                 // 1 <-> 2
77             }
```

```
            playing = askYesNo("Play again? (Y/N): ");
        }
        System.out.println("Thanks for playing!");
    }

    // ------------------------------------------------------------
    //  Game logic (one method per MIPS procedure)
    // ------------------------------------------------------------
    static void resetGame() {
        for (int i = 0; i < 9; i++) board[i] = 0;
        moveCount = 0;
    }

    /** Asks until the player types a legal, empty cell. Returns index 0..8. */
    static int askMove(int player) throws IOException {
        while (true) {
            System.out.print("Player " + (player == PLAYER_X ? "X" : "O") + ", choose a cell (1-9): ");
            String line = readLine();
            int value = parseInt(line);              // -1 if not a number
            if (value < 0) {
                System.out.println("  ! That is not a whole number. Enter a digit from 1 to 9.");
            } else if (value < 1 || value > 9) {
                System.out.println("  ! Out of range. Enter a digit from 1 to 9.");
            } else if (board[value - 1] != 0) {
                System.out.println("  ! That cell is already taken. Pick an empty one.");
            } else {
                return value - 1;
            }
        }
    }

    /** Returns the index of the winning line for player, or -1. */
    static int checkWin(int player) {
        for (int i = 0; i < WIN_LINES.length; i++) {
            int[] l = WIN_LINES[i];
            if (board[l[0]] == player && board[l[1]] == player && board[l[2]] == player) return i;
        }
        return -1;
    }

    static void highlightLine(int line, int player) {
        for (int cell : WIN_LINES[line]) drawMark(cell, player, COLOR_WIN);
    }

    static void printConsoleBoard() {
        StringBuilder sb = new StringBuilder("\n");
        for (int r = 0; r < 3; r++) {
            sb.append(' ').append(cellChar(r * 3)).append(" | ").append(cellChar(r * 3 + 1))
              .append(" | ").append(cellChar(r * 3 + 2)).append('\n');
            if (r < 2) sb.append("---+---+---\n");
        }
        System.out.println(sb);
    }

    static char cellChar(int idx) {
        if (board[idx] == PLAYER_X) return 'X';
        if (board[idx] == PLAYER_O) return 'O';
        return (char) ('1' + idx);
    }

    // ------------------------------------------------------------
    //  Drawing (same geometry as the .asm)
    // ------------------------------------------------------------
    static void drawBoardScreen() {
        display.fillScreen(COLOR_BG);
        display.drawText(10, 1, "TIC TAC TOE", COLOR_TEXT);
        display.drawRect(23, BOARD_Y, 1, 47, COLOR_GRID);
        display.drawRect(39, BOARD_Y, 1, 47, COLOR_GRID);
        display.drawRect(BOARD_X, 23, 47, 1, COLOR_GRID);
        display.drawRect(BOARD_X, 39, 47, 1, COLOR_GRID);
        for (int i = 0; i < 9; i++) {
            display.drawGlyph(cellX(i) + 1, cellY(i) + 1, (char) ('1' + i), COLOR_HINT);
        }
    }

    static int cellX(int idx) { return BOARD_X + (idx % 3) * CELL_PITCH; }
    static int cellY(int idx) { return BOARD_Y + (idx / 3) * CELL_PITCH; }

    static void drawMark(int cell, int player, int color) {
        int x0 = cellX(cell), y0 = cellY(cell);
        display.drawRect(x0, y0, CELL_SIZE, CELL_SIZE, COLOR_BG);  // erase hint digit
        if (player == PLAYER_X) drawXMark(x0, y0, color);
        else drawOMark(x0, y0, color);
    }

    static void drawXMark(int x0, int y0, int color) {
        for (int i = 0; i <= 10; i++) {
            display.drawPixel(x0 + 2 + i, y0 + 2 + i, color);
            display.drawPixel(x0 + 3 + i, y0 + 2 + i, color);
            display.drawPixel(x0 + 12 - i, y0 + 2 + i, color);
            display.drawPixel(x0 + 11 - i, y0 + 2 + i, color);
        }
    }

    static void drawOMark(int x0, int y0, int color) {
        display.drawRect(x0 + 2, y0 + 2, 11, 11, color);
        display.drawRect(x0 + 4, y0 + 4, 7, 7, COLOR_BG);
        display.drawPixel(x0 + 2, y0 + 2, COLOR_BG);
        display.drawPixel(x0 + 12, y0 + 2, COLOR_BG);
        display.drawPixel(x0 + 2, y0 + 12, COLOR_BG);
        display.drawPixel(x0 + 12, y0 + 12, COLOR_BG);
    }

    static void showStatus(String text) {
        display.drawRect(0, 57, 64, 7, COLOR_BG);
        display.drawText(4, 57, text, COLOR_TEXT);
    }

    // ------------------------------------------------------------
    //  Input helpers (read_line / parse_int in the .asm)
    // ------------------------------------------------------------
    static String readLine() throws IOException {
        String s = in.readLine();
        if (s == null) {                        // EOF: end cleanly instead of looping
            System.out.println("\nInput ended. Goodbye.");
            System.exit(0);
        }
        return s;
    }

    /** Non-negative integer with optional surrounding spaces, else -1. */
    static int parseInt(String s) {
        s = s.trim();
        if (s.isEmpty() || s.length() > 9) return -1;
        int v = 0;
        for (char c : s.toCharArray()) {
            if (c < '0' || c > '9') return -1;
            v = v * 10 + (c - '0');
        }
        return v;
    }

    static boolean askYesNo(String prompt) throws IOException {
        while (true) {
            System.out.print(prompt);
            String s = readLine().trim();
            if (s.equalsIgnoreCase("Y")) return true;
            if (s.equalsIgnoreCase("N")) return false;
            System.out.println("  ! Please answer Y or N.");
        }
    }

    // ------------------------------------------------------------
    //  Display: a software model of the MARS Bitmap Display
    // ------------------------------------------------------------
    static class Display {
        static final int W = 64, H = 64;
        final int[] pixels = new int[W * H];        // same layout as memory at 0x10008000
        final int background;

        // 3x5 font, 5 bytes per glyph, bit 2 = left pixel. Same table as the .asm.
        static final int[] FONT = {
            7,5,5,5,7, 2,6,2,2,7, 7,1,7,4,7, 7,1,7,1,7, 5,5,7,1,1,  // 0-4
            7,4,7,1,7, 7,4,7,5,7, 7,1,1,1,1, 7,5,7,5,7, 7,5,7,1,7,  // 5-9
            2,5,7,5,5, 6,5,6,5,6, 7,4,4,4,7, 6,5,5,5,6, 7,4,7,4,7,  // A-E
            7,4,7,4,4, 7,4,5,5,7, 5,5,7,5,5, 7,2,2,2,7, 1,1,1,5,7,  // F-J
            5,5,6,5,5, 4,4,4,4,7, 5,7,7,5,5, 6,5,5,5,5, 7,5,5,5,7,  // K-O
            7,5,7,4,4, 7,5,5,7,1, 6,5,6,5,5, 7,4,7,1,7, 7,2,2,2,2,  // P-T
            5,5,5,5,7, 5,5,5,5,2, 5,5,7,7,5, 5,5,2,5,5, 5,5,2,2,2,
```

```
        // U-Y
237           7,1,2,4,7, 0,0,0,0,0, 0,0,0,0,7, 7,1,3,0,2, 0,0,7,0,0,
        // Z, space, _, ?, -
238           0,2,0,2,0, 2,2,2,0,2
         // :, !
239        };
240
241        Display(int background) { this.background = background; fi
    llScreen(background); }
242
243        void fillScreen(int color) { java.util.Arrays.fill(pixels,
     color); }
244
245        void drawPixel(int x, int y, int color) {
246            if (x < 0 || y < 0 || x >= W || y >= H) return;   // i
    gnore out of range
247            pixels[y * W + x] = color;                           /
    / address = base + (y*64+x)*4
248        }
249
250        void drawRect(int x, int y, int w, int h, int color) {
251            for (int r = 0; r < h; r++)
252                for (int c = 0; c < w; c++) drawPixel(x + c, y + r
    , color);
253        }
254
255        static int glyphIndex(char ch) {
256            if (ch >= '0' && ch <= '9') return ch - '0';
257            if (ch >= 'A' && ch <= 'Z') return ch - 'A' + 10;
258            if (ch >= 'a' && ch <= 'z') return ch - 'a' + 10;
259            switch (ch) {
260                case ' ': return 36; case '_': return 37; case '-'
    : return 39;
261                case ':': return 40; case '!': return 41; default:
     return 38;   // '?'
262            }
263        }
264
265        void drawGlyph(int x, int y, char ch, int color) {
266            int base = glyphIndex(ch) * 5;
267            for (int row = 0; row < 5; row++) {
268                int bits = FONT[base + row];
269                for (int col = 0; col < 3; col++)
270                    if ((bits & (4 >> col)) != 0) drawPixel(x + co
    l, y + row, color);
271            }
272        }
273
274        void drawText(int x, int y, String s, int color) {
275            for (int i = 0; i < s.length(); i++) drawGlyph(x + 4 *
     i, y, s.charAt(i), color);
276        }
277
278        /** Prints the frame buffer as text: two pixel rows per li
    ne (# = both lit, " = top, . = bottom). */
279        void render() {
280            StringBuilder sb = new StringBuilder();
281            sb.append('+').append("-".repeat(W)).append("+\n");
282            for (int y = 0; y < H; y += 2) {
283                sb.append('|');
284                for (int x = 0; x < W; x++) {
285                    boolean top = pixels[y * W + x] != background;
286                    boolean bot = pixels[(y + 1) * W + x] != backg
    round;
287                    sb.append(top && bot ? '#' : top ? '"' : bot ?
     '.' : ' ');   // ASCII-safe
288                }
289                sb.append("|\n");
290            }
291            sb.append('+').append("-".repeat(W)).append("+\n");
292            System.out.print(sb);
293        }
294    }
295 }
296
```

Blackjack.java

```
  1 import java.io.BufferedReader;
  2 import java.io.IOException;
  3 import java.io.InputStreamReader;
  4 import java.util.Random;
  5
  6 /**
  7  * Blackjack.java -- Java planning / reference version of blackjac
        k.asm
  8  *
  9  * Module 4: MIPS Game Project (Game 2 of 3)
 10  * Author  : Dylan Flores
 11  *
 12  * Written first to plan the MIPS version. The deck and hands are
        plain int
 13  * arrays with explicit counters (exactly what the .asm does with
        byte arrays
 14  * and word counters), the card encoding is identical (rank = card
        % 13,
 15  * suit = card / 13), and the Display class models the MARS Bitmap
        Display
 16  * with the same card layout and 3x5 font.
 17  *
 18  * Run:  javac Blackjack.java && java Blackjack
 19  */
 20 public class Blackjack {
 21
 22     // ---- constants (mirror the .eqv values) ----
 23     static final int COLOR_FELT = 0x206838, COLOR_TEXT = 0xFFFFFF,
         COLOR_GOLD = 0xFFD040;
 24     static final int COLOR_CARD = 0xF8F8F0, COLOR_BACK = 0x2040A0,
         COLOR_BACK2 = 0x4878D0;
 25     static final int COLOR_RED = 0xD82020, COLOR_BLACK = 0x101010;
 26     static final int CARD_W = 9, CARD_H = 12, CARD_PITCH = 10, DEA
    LER_Y = 7, PLAYER_Y = 27;
 27     static final int MAX_HAND = 12;
 28     static final char[] SUIT_LETTERS = {'S', 'H', 'D', 'C'};
 29
 30     // ---- game state (mirrors .data) ----
 31     static final int[] deck = new int[52];
 32     static int deckPos = 0;
 33     static final int[] playerHand = new int[MAX_HAND];
 34     static final int[] dealerHand = new int[MAX_HAND];
 35     static int playerCount = 0, dealerCount = 0;
 36     static int wins = 0, losses = 0, pushes = 0;
 37     static String curMsg = "";
 38     static final Random rng = new Random();            // syscall
    40/42 in MIPS
 39     static final Display display = new Display(COLOR_FELT);
 40     static final BufferedReader in = new BufferedReader(new InputS
    treamReader(System.in));
 41
 42     public static void main(String[] args) throws IOException {
 43         System.out.print("=== BLACKJACK (Java reference) ===\n"
 44                 + "Get closer to 21 than the dealer without going
    over.\n"
 45                 + "Dealer hits on 16 and stands on 17. Aces count
    11 or 1.\n");
 46         do {
 47             playRound();
 48             System.out.println("Record: " + wins + " won, " + loss
    es + " lost, " + pushes + " pushed.");
 49         } while (askYesNo("Play another round? (Y/N): "));
 50         System.out.println("Thanks for playing!");
 51     }
 52
 53     // ------------------------------------------------------------
    -------
 54     //  One round (the round_start .. round_end section of main in
         the .asm)
 55     // ------------------------------------------------------------
    -------
 56     static void playRound() throws IOException {
 57         System.out.println("\n---------------- NEW ROUND ---------
    -------");
 58         shuffleDeck();
 59         playerCount = dealerCount = 0;
 60         dealCard(playerHand, true);
 61         dealCard(dealerHand, false);
 62         dealCard(playerHand, true);
 63         dealCard(dealerHand, false);
 64
 65         curMsg = "H HIT  S STAND";
 66         redrawTable(true);
 67         display.render();
 68         System.out.println("Dealer shows : " + cardName(dealerHand
    [0]) + " [??]");
 69         printPlayerLine();
 70
 71         // natural blackjack check
 72         if (handValue(playerHand, playerCount) == 21) {
 73             if (handValue(dealerHand, dealerCount) == 21) {
 74                 finish("PUSH", "\n*** Both have blackjack - push.
    ***", 0);
 75             } else {
```

```
            finish("BLACKJACK!", "\n*** BLACKJACK! You win! ***", 1);
        }
        printDealerFull();
        return;
    }

    // player's turn
    while (true) {
        System.out.print("(H)it or (S)tand? ");
        String s = readLine().trim().toUpperCase();
        char c = s.isEmpty() ? ' ' : s.charAt(0);
        if (c == 'H') {
            int card = dealCard(playerHand, true);
            System.out.println("You draw     : " + cardName(card));
            redrawTable(true);
            display.render();
            printPlayerLine();
            int total = handValue(playerHand, playerCount);
            if (total > 21) {
                finish("BUST", "\n*** BUST! You went over 21. Dealer wins. ***", -1);
                return;
            }
            if (total == 21) {
                System.out.println("You have 21 - standing automatically.");
                break;
            }
        } else if (c == 'S') {
            break;
        } else {
            System.out.println("  ! Please type H to hit or S to stand.");
        }
    }

    // dealer's turn: hit on 16 or less, stand on 17 or more
    redrawTable(false);
    System.out.println("Dealer reveals: " + handString(dealerHand, dealerCount));
    while (handValue(dealerHand, dealerCount) < 17) {
        int card = dealCard(dealerHand, false);
        System.out.println("Dealer hits  : " + cardName(card));
        redrawTable(false);
    }
    int d = handValue(dealerHand, dealerCount);
    int p = handValue(playerHand, playerCount);
    printDealerFull();
    if (d > 21)      finish("DEALER BUSTS", "\n*** Dealer busts! You win! ***", 1);
    else if (p > d)  finish("YOU WIN", "\n*** You win! ***", 1);
    else if (p < d)  finish("DEALER WINS", "\n*** Dealer wins. ***", -1);
    else             finish("PUSH", "\n*** Push - it's a tie. ***", 0);
}

/** Updates the record, the display message and prints the console result. */
static void finish(String displayMsg, String consoleMsg, int outcome) {
    if (outcome > 0) wins++; else if (outcome < 0) losses++; else pushes++;
    curMsg = displayMsg;
    redrawTable(false);
    display.render();
    System.out.println(consoleMsg);
}

// ---------------------------------------------------------------
//  Deck / hand procedures
// ---------------------------------------------------------------
static void shuffleDeck() {
    for (int i = 0; i < 52; i++) deck[i] = i;
    for (int i = 51; i > 0; i--) {                // Fisher-Yates
        int j = rng.nextInt(i + 1);
        int t = deck[i]; deck[i] = deck[j]; deck[j] = t;
    }
    deckPos = 0;
}

static int dealCard(int[] hand, boolean toPlayer) {
    if (deckPos >= 52) shuffleDeck();             // defensive: never happens in a round
    int card = deck[deckPos++];
    int count = toPlayer ? playerCount : dealerCount;
    if (count < MAX_HAND) {
        hand[count] = card;
        if (toPlayer) playerCount++; else dealerCount++;
    }
    return card;
}

static int cardValue(int card) {
    int rank = card % 13;
    if (rank == 0) return 11;                     // ace
    return Math.min(rank + 1, 10);                // 2..10, face cards = 10
}

static int handValue(int[] hand, int count) {
    int total = 0, aces = 0;
    for (int i = 0; i < count; i++) {
        int v = cardValue(hand[i]);
        total += v;
        if (v == 11) aces++;
    }
    while (total > 21 && aces > 0) { total -= 10; aces--; }
    return total;
}

static char rankChar(int rank) {
    if (rank == 0) return 'A';
    if (rank == 10) return 'J';
    if (rank == 11) return 'Q';
    if (rank == 12) return 'K';
    return (char) ('1' + rank);                   // 1..8 -> '2'..'9'
}

static String cardName(int card) {
    int rank = card % 13, suit = card / 13;
    return (rank == 9 ? "10" : String.valueOf(rankChar(rank))) + SUIT_LETTERS[suit];
}

static String handString(int[] hand, int count) {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < count; i++) sb.append(cardName(hand[i])).append(' ');
    return sb.toString();
}

static void printPlayerLine() {
    System.out.println("Your hand    : " + handString(playerHand, playerCount)
            + "   (total " + handValue(playerHand, playerCount) + ")");
}

static void printDealerFull() {
    System.out.println("Dealer hand  : " + handString(dealerHand, dealerCount)
            + "   (total " + handValue(dealerHand, dealerCount) + ")");
}

// ---------------------------------------------------------------
//  Table drawing (same geometry as the .asm)
// ---------------------------------------------------------------
static void redrawTable(boolean hideDealer) {
    display.fillScreen(COLOR_FELT);
    display.drawText(2, 1, "DEALER", COLOR_TEXT);
    drawHand(dealerHand, dealerCount, DEALER_Y, hideDealer);
    display.drawText(2, 21, "PLAYER", COLOR_TEXT);
    drawHand(playerHand, playerCount, PLAYER_Y, false);
    display.drawText(2, 41, "YOU", COLOR_TEXT);
    display.drawText(16, 41, "" + handValue(playerHand, playerCount), COLOR_GOLD);
    display.drawText(30, 41, "DLR", COLOR_TEXT);
    display.drawText(44, 41, hideDealer ? "?" : "" + handValue(dealerHand, dealerCount), COLOR_GOLD);
    display.drawText(2, 48, curMsg, COLOR_GOLD);
    display.drawText(2, 55, "W", COLOR_TEXT);  display.drawText(8, 55, "" + wins, COLOR_GOLD);
    display.drawText(22, 55, "L", COLOR_TEXT); display.drawText(28, 55, "" + losses, COLOR_GOLD);
    display.drawText(42, 55, "P", COLOR_TEXT); display.drawText(48, 55, "" + pushes, COLOR_GOLD);
}

static void drawHand(int[] hand, int count, int y, boolean hideSecond) {
    for (int i = 0; i < count; i++) {
        drawCard(2 + i * CARD_PITCH, y, hand[i], !(hideSecond && i == 1));
    }
}

static void drawCard(int x, int y, int card, boolean faceUp) {
    if (!faceUp) {
        display.drawRect(x, y, CARD_W, CARD_H, COLOR_BACK);
        display.drawRect(x + 2, y + 2, 5, 8, COLOR_BACK2);
```

```
            display.drawRect(x + 3, y + 3, 3, 6, COLOR_BACK);
            return;
        }
        display.drawRect(x, y, CARD_W, CARD_H, COLOR_CARD);
        int rank = card % 13, suit = card / 13;
        int ink = (suit == 1 || suit == 2) ? COLOR_RED : COLOR_BLACK;
        if (rank == 9) display.drawText(x + 1, y + 1, "10", ink);
        else display.drawGlyph(x + 1, y + 1, rankChar(rank), ink);
        display.drawGlyph(x + 5, y + 6, SUIT_LETTERS[suit], ink);
    }

    // ------------------------------------------------------------
    //  Input helpers
    // ------------------------------------------------------------
    static String readLine() throws IOException {
        String s = in.readLine();
        if (s == null) { System.out.println("\nInput ended. Goodbye."); System.exit(0); }
        return s;
    }

    static boolean askYesNo(String prompt) throws IOException {
        while (true) {
            System.out.print(prompt);
            String s = readLine().trim();
            if (s.equalsIgnoreCase("Y")) return true;
            if (s.equalsIgnoreCase("N")) return false;
            System.out.println("  ! Please answer Y or N.");
        }
    }
```

(Display class omitted, same as TicTacToe.java lines 220-293.)

```
}
```

Hangman.java

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.util.Random;

/**
 * Hangman.java -- Java planning / reference version of hangman.asm
 *
 * Module 4: MIPS Game Project (Game 3 of 3 - custom game)
 * Author  : Dylan Flores
 *
 * Written first to plan the MIPS version. The word table, the 26-entry
 * guessed[] flag array, the wrong-letter buffer and the six-part figure are
 * all the same as in hangman.asm, and the Display class models the MARS
 * Bitmap Display with the same geometry and 3x5 font.
 *
 * Run:  javac Hangman.java && java Hangman
 */
public class Hangman {

    static final int COLOR_BG = 0x181C28, COLOR_WOOD = 0xA0703A, COLOR_ROPE = 0xD0B070;
    static final int COLOR_BODY = 0xF0E0C0, COLOR_TEXT = 0xFFFFFF, COLOR_GOLD = 0xFFD040;
    static final int COLOR_BAD = 0xFF5050, COLOR_GOOD = 0x50FF70;
    static final int MAX_WRONG = 6, WORD_ROW_Y = 44, MISS_ROW_Y = 52;

    static final String[] WORD_TABLE = {
        "COMPUTER", "ASSEMBLY", "REGISTER", "PROCEDURE", "MEMORY", "BRANCH",
        "STACK", "SYSCALL", "PIPELINE", "COMPILER", "HARDWARE", "PROGRAM"
    };

    // ---- game state (mirrors .data) ----
    static String curWord = "";
    static final boolean[] guessed = new boolean[26];
    static final StringBuilder wrongBuf = new StringBuilder();
    static int wrongCount = 0, gamesWon = 0, gamesLost = 0;
    static final Random rng = new Random();
    static final Display display = new Display(COLOR_BG);
    static final BufferedReader in = new BufferedReader(new InputStreamReader(System.in));

    public static void main(String[] args) throws IOException {
        System.out.print("=== HANGMAN (Java reference) ===\n"
                + "I am thinking of a computer-architecture word.\n"
                + "Guess one letter at a time. You have 6 lives.\n\n");
        do {
            resetGame();
            drawScene();
            display.render();
            printState();
            while (true) {
                char letter = askLetter();                    // validated A-Z, not repeated
                guessed[letter - 'A'] = true;
                int n = countInWord(letter);
                if (n > 0) {
                    System.out.println("  Yes! '" + letter + "' appears " + n + " time(s).");
                } else {
                    wrongBuf.append(letter);
                    wrongCount++;
                    System.out.println("  No '" + letter + "' in the word.");
                }
                drawScene();
                printState();
                if (wordSolved()) {
                    gamesWon++;
                    display.drawText(30, 15, "YOU WIN", COLOR_GOOD);
                    display.render();
                    System.out.println("\n*** You got it! The word was " + curWord + ". ***");
                    break;
                }
                if (wrongCount >= MAX_WRONG) {
                    gamesLost++;
                    drawWordRow(true);                        // reveal missing letters in red
                    display.drawText(30, 15, "YOU LOSE", COLOR_BAD);
                    display.render();
                    System.out.println("\n*** Out of lives! The word was " + curWord + ". ***");
```

```
                    break;
                }
                display.render();
            }
            System.out.println("Score: " + gamesWon + " won, " + gamesLost + " lost.");
        } while (askYesNo("Play again? (Y/N): "));
        System.out.println("Thanks for playing!");
    }

    // ------------------------------------------------------------
    //  Game state procedures
    // ------------------------------------------------------------
    static void resetGame() {
        curWord = WORD_TABLE[rng.nextInt(WORD_TABLE.length)];
        java.util.Arrays.fill(guessed, false);
        wrongBuf.setLength(0);
        wrongCount = 0;
    }

    /** Asks until the input is exactly one letter that has not been tried. */
    static char askLetter() throws IOException {
        while (true) {
            System.out.print("Guess a letter: ");
            String s = readLine();
            if (s.length() != 1) { System.out.println("  ! Enter exactly one letter."); continue; }
            char c = Character.toUpperCase(s.charAt(0));
            if (c < 'A' || c > 'Z') { System.out.println("  ! That is not a letter (A-Z only)."); continue; }
            if (guessed[c - 'A'])   { System.out.println("  ! You already tried that letter."); continue; }
            return c;
        }
    }

    static int countInWord(char letter) {
        int n = 0;
        for (int i = 0; i < curWord.length(); i++) if (curWord.charAt(i) == letter) n++;
        return n;
    }

    static boolean wordSolved() {
        for (int i = 0; i < curWord.length(); i++) if (!guessed[curWord.charAt(i) - 'A']) return false;
        return true;
    }

    static void printState() {
        StringBuilder sb = new StringBuilder("Word : ");
        for (int i = 0; i < curWord.length(); i++) {
            char c = curWord.charAt(i);
            sb.append(guessed[c - 'A'] ? c : '_').append(' ');
        }
        sb.append("   Lives: ").append(MAX_WRONG - wrongCount).append("   Missed: ").append(wrongBuf);
        System.out.println(sb);
    }

    // ------------------------------------------------------------
    //  Scene drawing (same geometry as the .asm)
    // ------------------------------------------------------------
    static void drawScene() {
        display.fillScreen(COLOR_BG);
        display.drawText(30, 1, "HANGMAN", COLOR_GOLD);
        display.drawText(30, 8, "LIVES", COLOR_TEXT);
        display.drawText(54, 8, "" + (MAX_WRONG - wrongCount), COLOR_GOLD);
        drawGallows();
        drawFigure(wrongCount);
        drawWordRow(false);
        display.drawText(2, MISS_ROW_Y, "MISS", COLOR_TEXT);
        display.drawText(22, MISS_ROW_Y, wrongBuf.toString(), COLOR_BAD);
    }

    static void drawGallows() {
        display.drawRect(2, 38, 22, 1, COLOR_WOOD);     // base
        display.drawRect(5, 4, 1, 34, COLOR_WOOD);      // pole
        display.drawRect(5, 4, 13, 1, COLOR_WOOD);      // beam
        display.drawRect(17, 5, 1, 3, COLOR_ROPE);      // rope
    }

    /** Draws the first `parts` of: head, body, left arm, right arm, left leg, right leg. */
    static void drawFigure(int parts) {
        if (parts >= 1) {                               // head: 5x5 ring
            display.drawRect(15, 8, 5, 5, COLOR_BODY);
            display.drawRect(16, 9, 3, 3, COLOR_BG);
            display.drawPixel(15, 8, COLOR_BG);  display.drawPixel(19, 8, COLOR_BG);
            display.drawPixel(15, 12, COLOR_BG); display.drawPixel(19, 12, COLOR_BG);
        }
        if (parts >= 2) display.drawRect(17, 13, 1, 9, COLOR_BODY);   // body
        if (parts >= 3) drawDiag(16, 14, -1);  // left arm
        if (parts >= 4) drawDiag(18, 14, 1);   // right arm
        if (parts >= 5) drawDiag(16, 22, -1);  // left leg
        if (parts >= 6) drawDiag(18, 22, 1);   // right leg
    }

    static void drawDiag(int x, int y, int dx) {
        for (int i = 0; i < 4; i++) display.drawPixel(x + i * dx, y + i, COLOR_BODY);
    }

    static void drawWordRow(boolean revealAll) {
        for (int i = 0; i < curWord.length(); i++) {
            char c = curWord.charAt(i);
            int x = 2 + 4 * i;
            if (guessed[c - 'A'])      display.drawGlyph(x, WORD_ROW_Y, c, COLOR_TEXT);
            else if (revealAll)        display.drawGlyph(x, WORD_ROW_Y, c, COLOR_BAD);
            else                       display.drawGlyph(x, WORD_ROW_Y, '_', COLOR_GOLD);
        }
    }

    // ------------------------------------------------------------
    //  Input helpers
    // ------------------------------------------------------------
    static String readLine() throws IOException {
        String s = in.readLine();
        if (s == null) { System.out.println("\nInput ended. Goodbye."); System.exit(0); }
        return s;
    }

    static boolean askYesNo(String prompt) throws IOException {
        while (true) {
            System.out.print(prompt);
            String s = readLine().trim();
            if (s.equalsIgnoreCase("Y")) return true;
            if (s.equalsIgnoreCase("N")) return false;
            System.out.println("  ! Please answer Y or N.");
        }
    }
```

(Display class omitted, same as TicTacToe.java lines 220-293.)

```
}
```

MIPS source code

The graphics library and input library sections are identical in all three files, so they are listed in full in tictactoe.asm and left out of the other two to save space. The submitted .asm files contain them.

tictactoe.asm

```
################################################################################
#  tictactoe.asm  --  Two-player Tic-Tac-Toe for the MARS MIPS simulator
#
#  Module 4: MIPS Game Project  (Game 1 of 3)
#  Author : Dylan Flores
#
#  HOW TO RUN (MARS 4.5)
#    1. Open this file in MARS and assemble it (F3).
#    2. Tools -> Bitmap Display.  Set:
#         Unit Width  = 4      Unit Height = 4
#         Display Width = 256  Display Height = 256
#         Base address for display = 0x10008000 ($gp)
#       Click "Connect to MIPS".
#    3. Run (F5).  Type moves in the Run I/O console at the bottom of MARS.
#
#  GAME
#    Two players (X and O) take turns typing a cell number 1-9.
#    The board, marks, cell numbers and status text are drawn on the
#    64 x 64 pixel bitmap display; the console mirrors the board in text.
#    Invalid input (not a number, out of range, occupied cell) is rejected
#    with a message and the same player is asked again.
#
#  MEMORY MAP OF THE DISPLAY
#    64 x 64 pixels, one word (0x00RRGGBB) per pixel,
#    address(x, y) = 0x10008000 + (y * 64 + x) * 4
#
#  REGISTER / STACK CONVENTIONS USED
#    $a0-$a3 : arguments           $v0-$v1 : return values
#    $t0-$t9 : caller-saved temps  $s0-$s7 : callee-saved (pushed/popped)
#    draw_rect takes a 5th argument (color) on the stack at 0($sp).
################################################################################

        .eqv  DISPLAY_BASE  0x10008000
        .eqv  SCREEN_W      64
        .eqv  SCREEN_H      64
        .eqv  SCREEN_WORDS  4096          # 64 * 64

        .eqv  COLOR_BG      0x00202830    # dark slate background
        .eqv  COLOR_GRID    0x00E0E0E0    # light grey grid lines
        .eqv  COLOR_X       0x00FF5050    # red X
        .eqv  COLOR_O       0x0050A0FF    # blue O
        .eqv  COLOR_WIN     0x0050FF70    # green highlight for the winning line
        .eqv  COLOR_HINT    0x00606878    # dim cell numbers
        .eqv  COLOR_TEXT    0x00FFE060    # yellow status text

        .eqv  BOARD_X       8             # top-left pixel of the board
        .eqv  BOARD_Y       8
        .eqv  CELL_PITCH    16            # 15 pixel cell + 1 pixel line
        .eqv  CELL_SIZE     15

        .eqv  PLAYER_X      1
        .eqv  PLAYER_O      2

################################################################################
#  Small macros that wrap the print syscalls (procedures do the real work)
################################################################################
        .macro print_str(%label)
        la    $a0, %label
        li    $v0, 4
        syscall
        .end_macro

        .macro print_char(%reg)
        move  $a0, %reg
        li    $v0, 11
        syscall
        .end_macro

################################################################################
        .data
################################################################################
board:          .byte  0,0,0,0,0,0,0,0,0     # 0 = empty, 1 = X, 2 = O
                .align 2
win_lines:      .byte  0,1,2,  3,4,5,  6,7,8     # rows
                .byte  0,3,6,  1,4,7,  2,5,8     # columns
                .byte  0,4,8,  2,4,6             # diagonals
                .align 2
move_count:     .word  0
input_buf:      .space 64

# 3 x 5 pixel font.  5 bytes per glyph, one byte per row, bit2 = left pixel.
# Index: 0-9 digits, 10-35 letters A-Z, 36 ' ', 37 '_', 38 '?', 39 '-', 40 ':', 41 '!'
font:           .byte 7,5,5,5,7   # 0
                .byte 2,6,2,2,7   # 1
                .byte 7,1,7,4,7   # 2
                .byte 7,1,7,1,7   # 3
                .byte 5,5,7,1,1   # 4
                .byte 7,4,7,1,7   # 5
                .byte 7,4,7,5,7   # 6
                .byte 7,1,1,1,1   # 7
                .byte 7,5,7,5,7   # 8
                .byte 7,5,7,1,7   # 9
                .byte 2,5,7,5,5   # A
                .byte 6,5,6,5,6   # B
                .byte 7,4,4,4,7   # C
                .byte 6,5,5,5,6   # D
                .byte 7,4,7,4,7   # E
                .byte 7,4,7,4,4   # F
                .byte 7,4,5,5,7   # G
                .byte 5,5,7,5,5   # H
                .byte 7,2,2,2,7   # I
                .byte 1,1,1,5,7   # J
                .byte 5,5,6,5,5   # K
                .byte 4,4,4,4,7   # L
                .byte 5,7,7,5,5   # M
                .byte 6,5,5,5,5   # N
                .byte 7,5,5,5,7   # O
                .byte 7,5,7,4,4   # P
                .byte 7,5,5,7,1   # Q
                .byte 6,5,6,5,5   # R
                .byte 7,4,7,1,7   # S
                .byte 7,2,2,2,2   # T
                .byte 5,5,5,5,7   # U
                .byte 5,5,5,5,2   # V
                .byte 5,5,7,7,5   # W
                .byte 5,5,2,5,5   # X
                .byte 5,5,2,2,2   # Y
                .byte 7,1,2,4,7   # Z
                .byte 0,0,0,0,0   # space
                .byte 0,0,0,0,7   # _
                .byte 7,1,3,0,2   # ?
                .byte 0,0,7,0,0   # -
                .byte 0,2,0,2,0   # :
                .byte 2,2,2,0,2   # !
                .align 2

# ---- strings drawn on the bitmap display ----
title_txt:      .asciiz "TIC TAC TOE"
x_turn_txt:     .asciiz "X TURN"
o_turn_txt:     .asciiz "O TURN"
x_wins_txt:     .asciiz "X WINS!"
o_wins_txt:     .asciiz "O WINS!"
draw_txt:       .asciiz "DRAW"

# ---- console strings ----
welcome_msg:    .asciiz "=== TIC-TAC-TOE (MIPS) ===\nTwo players take turns. Type the number of an empty cell (1-9).\nWatch the Bitmap Display for the graphics; the board is also printed here.\n\n"
prompt_x:       .asciiz "Player X, choose a cell (1-9): "
prompt_o:       .asciiz "Player O, choose a cell (1-9): "
err_not_num:    .asciiz "  ! That is not a whole number. Enter a digit from 1 to 9.\n"
err_range:      .asciiz "  ! Out of range. Enter a digit from 1 to 9.\n"
err_taken:      .asciiz "  ! That cell is already taken. Pick an empty one.\n"
msg_x_wins:     .asciiz "\n*** Player X wins! ***\n"
msg_o_wins:     .asciiz "\n*** Player O wins! ***\n"
msg_draw:       .asciiz "\n*** It's a draw. ***\n"
again_prompt:   .asciiz "Play again? (Y/N): "
err_yn:         .asciiz "  ! Please answer Y or N.\n"
bye_msg:        .asciiz "Thanks for playing!\n"
row_sep:        .asciiz "---+---+---\n"
cell_sep:       .asciiz " | "
newline:        .asciiz "\n"
space1:         .asciiz " "

################################################################################
        .text
        .globl main
################################################################################
main:
        print_str(welcome_msg)

new_game:
        jal     reset_game              # clear board + move counter
        jal     draw_board_screen       # background, title, grid,
```

```
      hints
163         li      $s0, PLAYER_X           # $s0 = current player for
      the whole game
164
165 game_loop:
166         jal     print_console_board
167         move    $a0, $s0
168         jal     show_turn_status        # "X TURN" / "O TURN" on t
    he display
169
170 ask_move:
171         # ---- prompt the current player ----
172         li      $t0, PLAYER_X
173         bne     $s0, $t0, prompt_player_o
174         print_str(prompt_x)
175         j       read_move
176 prompt_player_o:
177         print_str(prompt_o)
178
179 read_move:
180         la      $a0, input_buf
181         li      $a1, 64
182         jal     read_line               # read one line into input
    _buf
183         la      $a0, input_buf
184         jal     parse_int               # $v0 = value, $v1 = 1 if
    valid number
185         beqz    $v1, bad_number
186         blt     $v0, 1, bad_range
187         bgt     $v0, 9, bad_range
188         addi    $s1, $v0, -1            # $s1 = cell index 0..8
189         lb      $t0, board($s1)
190         bnez    $t0, bad_taken
191
192         # ---- legal move: record it and draw it ----
193         sb      $s0, board($s1)
194         lw      $t0, move_count
195         addi    $t0, $t0, 1
196         sw      $t0, move_count
197
198         move    $a0, $s1                # cell
199         move    $a1, $s0                # player
200         jal     draw_mark
201
202         move    $a0, $s0
203         jal     check_win               # $v0 = winning line index
      or -1
204         bgez    $v0, player_won
205
206         lw      $t0, move_count
207         li      $t1, 9
208         beq     $t0, $t1, game_draw
209
210         # switch player: 1 -> 2, 2 -> 1
211         li      $t0, 3
212         sub     $s0, $t0, $s0
213         j       game_loop
214
215 bad_number:
216         print_str(err_not_num)
217         j       ask_move
218 bad_range:
219         print_str(err_range)
220         j       ask_move
221 bad_taken:
222         print_str(err_taken)
223         j       ask_move
224
225 player_won:
226         move    $a0, $v0                # line index
227         move    $a1, $s0                # player
228         jal     highlight_line
229         jal     print_console_board
230         li      $t0, PLAYER_X
231         bne     $s0, $t0, o_won
232         print_str(msg_x_wins)
233         la      $a0, x_wins_txt
234         jal     show_status
235         j       ask_again
236 o_won:
237         print_str(msg_o_wins)
238         la      $a0, o_wins_txt
239         jal     show_status
240         j       ask_again
241
242 game_draw:
243         jal     print_console_board
244         print_str(msg_draw)
245         la      $a0, draw_txt
246         jal     show_status
247
248 ask_again:
249         print_str(again_prompt)
250         la      $a0, input_buf
251         li      $a1, 64
252         jal     read_line
253         lb      $t0, input_buf
254         li      $t1, 'Y'
255         beq     $t0, $t1, new_game
256         li      $t1, 'y'
257         beq     $t0, $t1, new_game
258         li      $t1, 'N'
259         beq     $t0, $t1, exit_game
260         li      $t1, 'n'
261         beq     $t0, $t1, exit_game
262         print_str(err_yn)
263         j       ask_again
264
265 exit_game:
266         print_str(bye_msg)
267         li      $v0, 10
268         syscall
269
270 ################################################################
    ##############
271 #  reset_game -- clears the board array and the move counter
272 ################################################################
    ##############
273 reset_game:
274         la      $t0, board
275         li      $t1, 9
276 reset_loop:
277         sb      $zero, 0($t0)
278         addi    $t0, $t0, 1
279         addi    $t1, $t1, -1
280         bnez    $t1, reset_loop
281         sw      $zero, move_count
282         jr      $ra
283
284 ################################################################
    ##############
285 #  check_win(player) -> $v0 = index (0-7) of the winning line, or
     -1
286 ################################################################
    ##############
287 check_win:
288         li      $t0, 0                  # line index
289         la      $t1, win_lines
290 cw_loop:
291         lb      $t2, 0($t1)
292         lb      $t3, 1($t1)
293         lb      $t4, 2($t1)
294         lb      $t2, board($t2)
295         lb      $t3, board($t3)
296         lb      $t4, board($t4)
297         bne     $t2, $a0, cw_next
298         bne     $t3, $a0, cw_next
299         bne     $t4, $a0, cw_next
300         move    $v0, $t0                # all three belong to play
    er -> win
301         jr      $ra
302 cw_next:
303         addi    $t1, $t1, 3
304         addi    $t0, $t0, 1
305         blt     $t0, 8, cw_loop
306         li      $v0, -1
307         jr      $ra
308
309 ################################################################
    ##############
310 #  highlight_line(line, player) -- redraws the 3 winning marks in
     green
311 ################################################################
    ##############
312 highlight_line:
313         addi    $sp, $sp, -16
314         sw      $ra, 12($sp)
315         sw      $s2, 8($sp)
316         sw      $s3, 4($sp)
317         sw      $s4, 0($sp)
318         la      $s2, win_lines
319         sll     $t0, $a0, 1             # line * 3 = line*2 + line
320         add     $t0, $t0, $a0
321         add     $s2, $s2, $t0           # $s2 -> first cell of the
     line
322         move    $s3, $a1                # player
323         li      $s4, 3                  # 3 cells
324 hl_loop:
325         lb      $a0, 0($s2)
326         move    $a1, $s3
327         li      $a2, COLOR_WIN
328         jal     draw_mark_color
329         addi    $s2, $s2, 1
330         addi    $s4, $s4, -1
331         bnez    $s4, hl_loop
332         lw      $ra, 12($sp)
333         lw      $s2, 8($sp)
334         lw      $s3, 4($sp)
335         lw      $s4, 0($sp)
336         addi    $sp, $sp, 16
337         jr      $ra
338
```

```
339 ################################################################
      ##############
340 #  print_console_board -- prints the text version of the board
341 ################################################################
      ##############
342 print_console_board:
343         addi    $sp, $sp, -8
344         sw      $ra, 4($sp)
345         sw      $s5, 0($sp)
346         print_str(newline)
347         li      $s5, 0                  # cell index 0..8
348 pcb_row:
349         print_str(space1)
350         move    $a0, $s5
351         jal     print_cell
352         print_str(cell_sep)
353         addi    $a0, $s5, 1
354         jal     print_cell
355         print_str(cell_sep)
356         addi    $a0, $s5, 2
357         jal     print_cell
358         print_str(newline)
359         addi    $s5, $s5, 3
360         bge     $s5, 9, pcb_done
361         print_str(row_sep)
362         j       pcb_row
363 pcb_done:
364         print_str(newline)
365         lw      $ra, 4($sp)
366         lw      $s5, 0($sp)
367         addi    $sp, $sp, 8
368         jr      $ra
369
370 # print_cell(index): prints 'X', 'O' or the cell number if empty
371 print_cell:
372         lb      $t0, board($a0)
373         beqz    $t0, pc_empty
374         li      $t1, 'X'
375         li      $t2, PLAYER_X
376         beq     $t0, $t2, pc_print
377         li      $t1, 'O'
378         j       pc_print
379 pc_empty:
380         addi    $t1, $a0, '1'           # '1' + index
381 pc_print:
382         print_char($t1)
383         jr      $ra
384
385 ################################################################
      ##############
386 #  show_turn_status(player) -- writes "X TURN" or "O TURN" on the
      display
387 ################################################################
      ##############
388 show_turn_status:
389         li      $t0, PLAYER_X
390         bne     $a0, $t0, sts_o
391         la      $a0, x_turn_txt
392         j       show_status
393 sts_o:
394         la      $a0, o_turn_txt
395         # fall through into show_status
396
397 ################################################################
      ##############
398 #  show_status(string) -- clears the status strip and draws the te
      xt
399 ################################################################
      ##############
400 show_status:
401         addi    $sp, $sp, -8
402         sw      $ra, 4($sp)
403         sw      $s6, 0($sp)
404         move    $s6, $a0
405         li      $a0, 0                  # clear the strip y = 57..
      63
406         li      $a1, 57
407         li      $a2, SCREEN_W
408         li      $a3, 7
409         li      $t0, COLOR_BG
410         addi    $sp, $sp, -4
411         sw      $t0, 0($sp)
412         jal     draw_rect
413         addi    $sp, $sp, 4
414         li      $a0, 4
415         li      $a1, 57
416         move    $a2, $s6
417         li      $a3, COLOR_TEXT
418         jal     draw_text
419         lw      $ra, 4($sp)
420         lw      $s6, 0($sp)
421         addi    $sp, $sp, 8
422         jr      $ra
423
424 ################################################################
      ##############
425 #  draw_board_screen -- background, title, grid lines, dim cell nu
      mbers
426 ################################################################
      ##############
427 draw_board_screen:
428         addi    $sp, $sp, -8
429         sw      $ra, 4($sp)
430         sw      $s7, 0($sp)
431
432         li      $a0, COLOR_BG
433         jal     fill_screen
434
435         li      $a0, 10
436         li      $a1, 1
437         la      $a2, title_txt
438         li      $a3, COLOR_TEXT
439         jal     draw_text
440
441         # two vertical grid lines (1 x 47) at x = 23 and x = 39
442         li      $a0, 23
443         li      $a1, BOARD_Y
444         li      $a2, 1
445         li      $a3, 47
446         li      $t0, COLOR_GRID
447         addi    $sp, $sp, -4
448         sw      $t0, 0($sp)
449         jal     draw_rect
450         addi    $sp, $sp, 4
451
452         li      $a0, 39
453         li      $a1, BOARD_Y
454         li      $a2, 1
455         li      $a3, 47
456         li      $t0, COLOR_GRID
457         addi    $sp, $sp, -4
458         sw      $t0, 0($sp)
459         jal     draw_rect
460         addi    $sp, $sp, 4
461
462         # two horizontal grid lines (47 x 1) at y = 23 and y = 39
463         li      $a0, BOARD_X
464         li      $a1, 23
465         li      $a2, 47
466         li      $a3, 1
467         li      $t0, COLOR_GRID
468         addi    $sp, $sp, -4
469         sw      $t0, 0($sp)
470         jal     draw_rect
471         addi    $sp, $sp, 4
472
473         li      $a0, BOARD_X
474         li      $a1, 39
475         li      $a2, 47
476         li      $a3, 1
477         li      $t0, COLOR_GRID
478         addi    $sp, $sp, -4
479         sw      $t0, 0($sp)
480         jal     draw_rect
481         addi    $sp, $sp, 4
482
483         # dim cell numbers 1..9 in the top-left corner of each cel
      l
484         li      $s7, 0
485 dbs_hint:
486         move    $a0, $s7
487         jal     cell_origin             # $v0 = x0, $v1 = y0
488         addi    $a0, $v0, 1
489         addi    $a1, $v1, 1
490         addi    $a2, $s7, '1'
491         li      $a3, COLOR_HINT
492         jal     draw_glyph
493         addi    $s7, $s7, 1
494         blt     $s7, 9, dbs_hint
495
496         lw      $ra, 4($sp)
497         lw      $s7, 0($sp)
498         addi    $sp, $sp, 8
499         jr      $ra
500
501 ################################################################
      ##############
502 #  cell_origin(index) -> $v0 = x0, $v1 = y0 (top-left pixel of the
       cell)
503 ################################################################
      ##############
504 cell_origin:
505         li      $t0, 3
506         div     $a0, $t0
507         mflo    $t1                     # row
508         mfhi    $t2                     # column
509         mul     $v0, $t2, CELL_PITCH
510         addi    $v0, $v0, BOARD_X
511         mul     $v1, $t1, CELL_PITCH
512         addi    $v1, $v1, BOARD_Y
513         jr      $ra
514
```

```
################################################################################
#  draw_mark(cell, player) -- draws X in red or O in blue
#  draw_mark_color(cell, player, color) -- same but with an explicit color
################################################################################
draw_mark:
        li      $a2, COLOR_X
        li      $t0, PLAYER_X
        beq     $a1, $t0, draw_mark_color
        li      $a2, COLOR_O
draw_mark_color:
        addi    $sp, $sp, -20
        sw      $ra, 16($sp)
        sw      $s0, 12($sp)
        sw      $s1, 8($sp)
        sw      $s2, 4($sp)
        sw      $s3, 0($sp)
        move    $s2, $a1                # player
        move    $s3, $a2                # color
        jal     cell_origin
        move    $s0, $v0                # x0
        move    $s1, $v1                # y0

        # erase the cell interior (removes the hint digit)
        move    $a0, $s0
        move    $a1, $s1
        li      $a2, CELL_SIZE
        li      $a3, CELL_SIZE
        li      $t0, COLOR_BG
        addi    $sp, $sp, -4
        sw      $t0, 0($sp)
        jal     draw_rect
        addi    $sp, $sp, 4

        move    $a0, $s0
        move    $a1, $s1
        move    $a2, $s3
        li      $t0, PLAYER_X
        bne     $s2, $t0, dmc_o
        jal     draw_x_mark
        j       dmc_done
dmc_o:
        jal     draw_o_mark
dmc_done:
        lw      $ra, 16($sp)
        lw      $s0, 12($sp)
        lw      $s1, 8($sp)
        lw      $s2, 4($sp)
        lw      $s3, 0($sp)
        addi    $sp, $sp, 20
        jr      $ra

# draw_x_mark(x0, y0, color): two 2-pixel-thick diagonals inside the cell
draw_x_mark:
        addi    $sp, $sp, -20
        sw      $ra, 16($sp)
        sw      $s0, 12($sp)
        sw      $s1, 8($sp)
        sw      $s2, 4($sp)
        sw      $s3, 0($sp)
        move    $s0, $a0
        move    $s1, $a1
        move    $s2, $a2
        li      $s3, 0                  # i = 0..10
dx_loop:
        addi    $a0, $s0, 2
        add     $a0, $a0, $s3           # x0 + 2 + i
        addi    $a1, $s1, 2
        add     $a1, $a1, $s3           # y0 + 2 + i
        move    $a2, $s2
        jal     draw_pixel
        addi    $a0, $s0, 3
        add     $a0, $a0, $s3           # x0 + 3 + i   (thickness)
        addi    $a1, $s1, 2
        add     $a1, $a1, $s3
        move    $a2, $s2
        jal     draw_pixel
        addi    $a0, $s0, 12
        sub     $a0, $a0, $s3           # x0 + 12 - i
        addi    $a1, $s1, 2
        add     $a1, $a1, $s3
        move    $a2, $s2
        jal     draw_pixel
        addi    $a0, $s0, 11
        sub     $a0, $a0, $s3           # x0 + 11 - i
        addi    $a1, $s1, 2
        add     $a1, $a1, $s3
        move    $a2, $s2
        jal     draw_pixel
        addi    $s3, $s3, 1
        ble     $s3, 10, dx_loop
        lw      $ra, 16($sp)
        lw      $s0, 12($sp)
        lw      $s1, 8($sp)
        lw      $s2, 4($sp)
        lw      $s3, 0($sp)
        addi    $sp, $sp, 20
        jr      $ra

# draw_o_mark(x0, y0, color): 11x11 ring, 2 pixels thick, rounded corners
draw_o_mark:
        addi    $sp, $sp, -16
        sw      $ra, 12($sp)
        sw      $s0, 8($sp)
        sw      $s1, 4($sp)
        sw      $s2, 0($sp)
        move    $s0, $a0
        move    $s1, $a1
        move    $s2, $a2

        addi    $a0, $s0, 2
        addi    $a1, $s1, 2
        li      $a2, 11
        li      $a3, 11
        addi    $sp, $sp, -4
        sw      $s2, 0($sp)
        jal     draw_rect               # outer square in the mark color
        addi    $sp, $sp, 4

        addi    $a0, $s0, 4
        addi    $a1, $s1, 4
        li      $a2, 7
        li      $a3, 7
        li      $t0, COLOR_BG
        addi    $sp, $sp, -4
        sw      $t0, 0($sp)
        jal     draw_rect               # hollow centre
        addi    $sp, $sp, 4

        # knock out the four corners so the ring looks round
        addi    $a0, $s0, 2
        addi    $a1, $s1, 2
        li      $a2, COLOR_BG
        jal     draw_pixel
        addi    $a0, $s0, 12
        addi    $a1, $s1, 2
        li      $a2, COLOR_BG
        jal     draw_pixel
        addi    $a0, $s0, 2
        addi    $a1, $s1, 12
        li      $a2, COLOR_BG
        jal     draw_pixel
        addi    $a0, $s0, 12
        addi    $a1, $s1, 12
        li      $a2, COLOR_BG
        jal     draw_pixel

        lw      $ra, 12($sp)
        lw      $s0, 8($sp)
        lw      $s1, 4($sp)
        lw      $s2, 0($sp)
        addi    $sp, $sp, 16
        jr      $ra

################################################################################
#  ======================  GRAPHICS LIBRARY  ======================
################################################################################

# fill_screen(color): writes color to all 4096 display words
fill_screen:
        li      $t0, DISPLAY_BASE
        li      $t1, SCREEN_WORDS
fs_loop:
        sw      $a0, 0($t0)
        addi    $t0, $t0, 4
        addi    $t1, $t1, -1
        bnez    $t1, fs_loop
        jr      $ra

# draw_pixel(x, y, color): sets one pixel; silently ignores out-of-range coords
draw_pixel:
        bltz    $a0, dp_done
        bltz    $a1, dp_done
        bge     $a0, SCREEN_W, dp_done
        bge     $a1, SCREEN_H, dp_done
        sll     $t0, $a1, 6             # y * 64
        add     $t0, $t0, $a0           # + x
        sll     $t0, $t0, 2             # * 4 bytes per pixel
        li      $t1, DISPLAY_BASE
        add     $t0, $t0, $t1
        sw      $a2, 0($t0)
dp_done:
        jr      $ra
```

```
697
698 # draw_rect(x, y, w, h, color-on-stack): filled rectangle.
699 # The 5th argument is read from 0($sp) BEFORE the prologue moves $
      sp.
700 draw_rect:
701         lw      $t9, 0($sp)             # color (5th argument)
702         addi    $sp, $sp, -28
703         sw      $ra, 24($sp)
704         sw      $s0, 20($sp)
705         sw      $s1, 16($sp)
706         sw      $s2, 12($sp)
707         sw      $s3, 8($sp)
708         sw      $s4, 4($sp)
709         sw      $s5, 0($sp)
710         move    $s0, $a0                # x
711         move    $s1, $a1                # y
712         move    $s2, $a2                # w
713         move    $s3, $a3                # h
714         move    $s4, $t9                # color
715         li      $s5, 0                  # row
716 dr_row:
717         bge     $s5, $s3, dr_done
718         li      $t8, 0                  # column (kept in $t8; dra
      w_pixel never touches it)
719 dr_col:
720         bge     $t8, $s2, dr_next_row
721         add     $a0, $s0, $t8
722         add     $a1, $s1, $s5
723         move    $a2, $s4
724         jal     draw_pixel
725         addi    $t8, $t8, 1
726         j       dr_col
727 dr_next_row:
728         addi    $s5, $s5, 1
729         j       dr_row
730 dr_done:
731         lw      $ra, 24($sp)
732         lw      $s0, 20($sp)
733         lw      $s1, 16($sp)
734         lw      $s2, 12($sp)
735         lw      $s3, 8($sp)
736         lw      $s4, 4($sp)
737         lw      $s5, 0($sp)
738         addi    $sp, $sp, 28
739         jr      $ra
740
741 # glyph_index(char) -> $v0 = index into the font table
742 glyph_index:
743         blt     $a0, '0', gi_not_digit
744         bgt     $a0, '9', gi_not_digit
745         addi    $v0, $a0, -48           # '0'..'9' -> 0..9
746         jr      $ra
747 gi_not_digit:
748         blt     $a0, 'A', gi_not_upper
749         bgt     $a0, 'Z', gi_not_upper
750         addi    $v0, $a0, -55           # 'A' (65) -> 10
751         jr      $ra
752 gi_not_upper:
753         blt     $a0, 'a', gi_symbol
754         bgt     $a0, 'z', gi_symbol
755         addi    $v0, $a0, -87           # 'a' (97) -> 10
756         jr      $ra
757 gi_symbol:
758         li      $v0, 36
759         beq     $a0, ' ', gi_ret
760         li      $v0, 37
761         beq     $a0, '_', gi_ret
762         li      $v0, 39
763         beq     $a0, '-', gi_ret
764         li      $v0, 40
765         beq     $a0, ':', gi_ret
766         li      $v0, 41
767         beq     $a0, '!', gi_ret
768         li      $v0, 38                 # anything else prints as
      '?'
769 gi_ret:
770         jr      $ra
771
772 # draw_glyph(x, y, char, color): draws one 3x5 character
773 draw_glyph:
774         addi    $sp, $sp, -28
775         sw      $ra, 24($sp)
776         sw      $s0, 20($sp)
777         sw      $s1, 16($sp)
778         sw      $s2, 12($sp)
779         sw      $s3, 8($sp)
780         sw      $s4, 4($sp)
781         sw      $s5, 0($sp)
782         move    $s0, $a0                # x
783         move    $s1, $a1                # y
784         move    $s3, $a3                # color
785         move    $a0, $a2
786         jal     glyph_index
787         la      $s2, font
788         sll     $t0, $v0, 2             # index * 5 = index*4 + in
      dex
789         add     $t0, $t0, $v0
790         add     $s2, $s2, $t0           # $s2 -> first row byte
791         li      $s4, 0                  # row 0..4
792 dg_row:
793         lb      $s5, 0($s2)             # row bits: bit2 bit1 bit0
       = left..right
794         li      $t8, 0                  # column 0..2 (kept in $t8
      )
795 dg_col:
796         li      $t1, 4
797         srlv    $t1, $t1, $t8           # mask = 4 >> col
798         and     $t1, $t1, $s5
799         beqz    $t1, dg_skip
800         add     $a0, $s0, $t8
801         add     $a1, $s1, $s4
802         move    $a2, $s3
803         jal     draw_pixel
804 dg_skip:
805         addi    $t8, $t8, 1
806         blt     $t8, 3, dg_col
807         addi    $s2, $s2, 1
808         addi    $s4, $s4, 1
809         blt     $s4, 5, dg_row
810         lw      $ra, 24($sp)
811         lw      $s0, 20($sp)
812         lw      $s1, 16($sp)
813         lw      $s2, 12($sp)
814         lw      $s3, 8($sp)
815         lw      $s4, 4($sp)
816         lw      $s5, 0($sp)
817         addi    $sp, $sp, 28
818         jr      $ra
819
820 # draw_text(x, y, string, color): draws a NUL-terminated string, 4
       px per char
821 draw_text:
822         addi    $sp, $sp, -20
823         sw      $ra, 16($sp)
824         sw      $s0, 12($sp)
825         sw      $s1, 8($sp)
826         sw      $s2, 4($sp)
827         sw      $s3, 0($sp)
828         move    $s0, $a0
829         move    $s1, $a1
830         move    $s2, $a2
831         move    $s3, $a3
832 dt_loop:
833         lb      $a2, 0($s2)
834         beqz    $a2, dt_done
835         move    $a0, $s0
836         move    $a1, $s1
837         move    $a3, $s3
838         jal     draw_glyph
839         addi    $s0, $s0, 4
840         addi    $s2, $s2, 1
841         j       dt_loop
842 dt_done:
843         lw      $ra, 16($sp)
844         lw      $s0, 12($sp)
845         lw      $s1, 8($sp)
846         lw      $s2, 4($sp)
847         lw      $s3, 0($sp)
848         addi    $sp, $sp, 20
849         jr      $ra
850
851 ################################################################
      ###############
852 #  =====================  INPUT LIBRARY  ======================
853 ################################################################
      ###############
854
855 # read_line(buffer, size): reads one line (syscall 8) and strips t
      he newline.
856 # Returns $v0 = number of characters kept.
857 read_line:
858         move    $t0, $a0                # remember buffer start
859         li      $v0, 8
860         syscall
861         li      $v0, 0
862 rl_scan:
863         lb      $t1, 0($t0)
864         beqz    $t1, rl_done
865         li      $t2, '\n'
866         beq     $t1, $t2, rl_strip
867         li      $t2, '\r'
868         beq     $t1, $t2, rl_strip
869         addi    $t0, $t0, 1
870         addi    $v0, $v0, 1
871         j       rl_scan
872 rl_strip:
873         sb      $zero, 0($t0)
874 rl_done:
875         jr      $ra
876
877 # parse_int(buffer) -> $v0 = value, $v1 = 1 if the whole line was
      a valid
```

```
878 # non-negative integer (surrounding spaces allowed), else $v1 = 0.
879 parse_int:
880         li      $v0, 0
881         li      $v1, 0
882         li      $t3, 0                  # digit count
883 pi_skip_lead:
884         lb      $t0, 0($a0)
885         li      $t1, ' '
886         bne     $t0, $t1, pi_digits
887         addi    $a0, $a0, 1
888         j       pi_skip_lead
889 pi_digits:
890         lb      $t0, 0($a0)
891         beqz    $t0, pi_end
892         li      $t1, ' '
893         beq     $t0, $t1, pi_trail
894         blt     $t0, '0', pi_bad
895         bgt     $t0, '9', pi_bad
896         addi    $t0, $t0, -48
897         mul     $v0, $v0, 10
898         add     $v0, $v0, $t0
899         addi    $t3, $t3, 1
900         bgt     $t3, 9, pi_bad          # absurdly long number ->
      reject (no overflow)
901         addi    $a0, $a0, 1
902         j       pi_digits
903 pi_trail:                               # only spaces may follow t
      he digits
904         lb      $t0, 0($a0)
905         beqz    $t0, pi_end
906         li      $t1, ' '
907         bne     $t0, $t1, pi_bad
908         addi    $a0, $a0, 1
909         j       pi_trail
910 pi_end:
911         beqz    $t3, pi_bad             # no digits at all (empty
      line)
912         li      $v1, 1
913         jr      $ra
914 pi_bad:
915         li      $v0, -1
916         li      $v1, 0
917         jr      $ra
918
```

blackjack.asm

```
  1 ################################################################
      ##############
  2 #  blackjack.asm  --  Blackjack (player vs. dealer) for the MARS M
      IPS simulator
  3 #
  4 #  Module 4: MIPS Game Project  (Game 2 of 3)
  5 #  Author : Dylan Flores
  6 #
  7 #  HOW TO RUN (MARS 4.5)
  8 #    1. Open this file in MARS and assemble it (F3).
  9 #    2. Tools -> Bitmap Display.  Set:
 10 #         Unit Width  = 4      Unit Height = 4
 11 #         Display Width = 256  Display Height = 256
 12 #         Base address for display = 0x10008000 ($gp)
 13 #       Click "Connect to MIPS".
 14 #    3. Run (F5).  Answer the prompts in the Run I/O console.
 15 #
 16 #  GAME
 17 #    Standard blackjack against a dealer that must hit on 16 and s
      tand on 17.
 18 #    A 52-card deck is stored in a byte array and shuffled (Fisher
      -Yates) with
 19 #    the MARS random syscall. Cards are drawn on the bitmap displa
      y (rank and
 20 #    suit letter, red for hearts/diamonds, black for spades/clubs)
      ; the
 21 #    dealer's second card is face down until the player stands.
 22 #    Aces count 11 and automatically drop to 1 when a hand would b
      ust.
 23 #    The console mirrors every hand and result in text.
 24 #
 25 #  CARD ENCODING  (0..51)
 26 #    rank = card % 13   (0 = Ace, 1..8 = 2..9, 9 = 10, 10 = J, 11
      = Q, 12 = K)
 27 #    suit = card / 13   (0 = Spades, 1 = Hearts, 2 = Diamonds, 3 =
       Clubs)
 28 #
 29 #  REGISTER / STACK CONVENTIONS USED
 30 #    $a0-$a3 arguments, $v0-$v1 results, $t* caller-saved, $s* cal
      lee-saved
 31 #    and saved on the stack by every procedure that uses them.
 32 #    draw_rect takes a 5th argument (color) on the stack at 0($sp)
      .
 33 ################################################################
      ##############
 34
 35         .eqv  DISPLAY_BASE  0x10008000
 36         .eqv  SCREEN_W      64
 37         .eqv  SCREEN_H      64
 38         .eqv  SCREEN_WORDS  4096
 39
 40         .eqv  COLOR_FELT    0x00206838    # green table felt
 41         .eqv  COLOR_TEXT    0x00FFFFFF    # white labels
 42         .eqv  COLOR_GOLD    0x00FFD040    # yellow status text
 43         .eqv  COLOR_CARD    0x00F8F8F0    # card face
 44         .eqv  COLOR_BACK    0x002040A0    # card back (dark blue)
 45         .eqv  COLOR_BACK2   0x004878D0    # card back pattern (lig
      ht blue)
 46         .eqv  COLOR_RED     0x00D82020    # hearts / diamonds
 47         .eqv  COLOR_BLACK   0x00101010    # spades / clubs
 48
 49         .eqv  CARD_W        9
 50         .eqv  CARD_H        12
 51         .eqv  CARD_PITCH    10
 52         .eqv  DEALER_Y      7
 53         .eqv  PLAYER_Y      27
 54         .eqv  MAX_HAND      12
 55
 56         .macro print_str(%label)
 57         la    $a0, %label
 58         li    $v0, 4
 59         syscall
 60         .end_macro
 61
 62         .macro print_int(%reg)
 63         move  $a0, %reg
 64         li    $v0, 1
 65         syscall
 66         .end_macro
 67
 68         .macro print_char(%reg)
 69         move  $a0, %reg
 70         li    $v0, 11
 71         syscall
 72         .end_macro
 73
 74 ################################################################
      ##############
 75         .data
 76 ################################################################
      ##############
 77 deck:           .space 52              # 52 card codes, shuffled
      each round
 78 deck_pos:       .word  0               # next card to deal
```

```
 79 player_hand:    .space MAX_HAND
 80 dealer_hand:    .space MAX_HAND
 81 player_count:   .word  0
 82 dealer_count:   .word  0
 83 wins:           .word  0
 84 losses:         .word  0
 85 pushes:         .word  0
 86 cur_msg:        .word  0               # pointer to the message s
        hown on row 48
 87 input_buf:      .space 64
 88 num_buf:        .space 12              # scratch for int_to_str
 89
 90 # 3 x 5 pixel font (see tictactoe.asm for the index layout)
 91 font:           .byte 7,5,5,5,7, 2,6,2,2,7, 7,1,7,4,7, 7,1,7,1,7,
        5,5,7,1,1
 92                 .byte 7,4,7,1,7, 7,4,7,5,7, 7,1,1,1,1, 7,5,7,5,7,
        7,5,7,1,7
 93                 .byte 2,5,7,5,5, 6,5,6,5,6, 7,4,4,4,7, 6,5,5,5,6,
        7,4,7,4,7
 94                 .byte 7,4,7,4,4, 7,4,5,5,7, 5,5,7,5,5, 7,2,2,2,7,
        1,1,1,5,7
 95                 .byte 5,5,6,5,5, 4,4,4,4,7, 5,7,7,5,5, 6,5,5,5,5,
        7,5,5,5,7
 96                 .byte 7,5,7,4,4, 7,5,5,7,1, 6,5,6,5,5, 7,4,7,1,7,
        7,2,2,2,2
 97                 .byte 5,5,5,5,7, 5,5,5,5,2, 5,5,7,7,5, 5,5,2,5,5,
        5,5,2,2,2
 98                 .byte 7,1,2,4,7, 0,0,0,0,0, 0,0,0,0,7, 7,1,3,0,2,
        0,0,7,0,0
 99                 .byte 0,2,0,2,0, 2,2,2,0,2
100                 .align 2
101
102 suit_letters:   .byte 'S','H','D','C'
103                 .align 2
104
105 # ---- strings drawn on the display ----
106 dealer_lbl:     .asciiz "DEALER"
107 player_lbl:     .asciiz "PLAYER"
108 you_lbl:        .asciiz "YOU"
109 dlr_lbl:        .asciiz "DLR"
110 w_lbl:          .asciiz "W"
111 l_lbl:          .asciiz "L"
112 p_lbl:          .asciiz "P"
113 unknown_lbl:    .asciiz "?"
114 msg_hit_stand:  .asciiz "H HIT  S STAND"
115 msg_blackjack:  .asciiz "BLACKJACK!"
116 msg_you_win:    .asciiz "YOU WIN"
117 msg_dealer_win: .asciiz "DEALER WINS"
118 msg_push:       .asciiz "PUSH"
119 msg_bust:       .asciiz "BUST"
120 msg_dealer_bust:.asciiz "DEALER BUSTS"
121
122 # ---- console strings ----
123 welcome:        .asciiz "=== BLACKJACK (MIPS) ===\nGet closer to 2
        1 than the dealer without going over.\nDealer hits on 16 and stand
        s on 17. Aces count 11 or 1.\n\n"
124 round_hdr:      .asciiz "\n---------------- NEW ROUND ------------
        ----\n"
125 c_dealer_shows: .asciiz "Dealer shows : "
126 c_hidden:       .asciiz " [??]"
127 c_your_hand:    .asciiz "Your hand    : "
128 c_total:        .asciiz "   (total "
129 c_close:        .asciiz ")\n"
130 c_prompt:       .asciiz "(H)it or (S)tand? "
131 c_bad_hs:       .asciiz "  ! Please type H to hit or S to stand.\n
        "
132 c_you_draw:     .asciiz "You draw     : "
133 c_21_stand:     .asciiz "You have 21 - standing automatically.\n"
134 c_bust:         .asciiz "\n*** BUST! You went over 21. Dealer wins
        . ***\n"
135 c_dealer_hand:  .asciiz "Dealer hand  : "
136 c_dealer_reveal:.asciiz "Dealer reveals: "
137 c_dealer_hits:  .asciiz "Dealer hits  : "
138 c_dealer_bust:  .asciiz "\n*** Dealer busts! You win! ***\n"
139 c_you_win:     .asciiz "\n*** You win! ***\n"
140 c_dealer_wins:  .asciiz "\n*** Dealer wins. ***\n"
141 c_push:        .asciiz "\n*** Push - it's a tie. ***\n"
142 c_blackjack:   .asciiz "\n*** BLACKJACK! You win! ***\n"
143 c_both_bj:     .asciiz "\n*** Both have blackjack - push. ***\n"
144 c_record1:     .asciiz "Record: "
145 c_record2:     .asciiz " won, "
146 c_record3:     .asciiz " lost, "
147 c_record4:     .asciiz " pushed.\n"
148 c_again:       .asciiz "Play another round? (Y/N): "
149 c_bad_yn:      .asciiz "  ! Please answer Y or N.\n"
150 c_bye:         .asciiz "Thanks for playing!\n"
151 newline:       .asciiz "\n"
152 space1:        .asciiz " "
153 rank_10:       .asciiz "10"
154
155 ################################################################
        ###############
156         .text
157         .globl main
158 ################################################################
        ###############
159 main:
160         print_str(welcome)
161         # seed the random generator (id 0) from the system clock
162         li      $v0, 30
163         syscall                         # $a0 = low 32 bits of tim
        e
164         move    $a1, $a0
165         li      $a0, 0
166         li      $v0, 40
167         syscall
168
169 round_start:
170         print_str(round_hdr)
171         jal     shuffle_deck
172         sw      $zero, player_count
173         sw      $zero, dealer_count
174
175         la      $a0, player_hand
176         la      $a1, player_count
177         jal     deal_card
178         la      $a0, dealer_hand
179         la      $a1, dealer_count
180         jal     deal_card
181         la      $a0, player_hand
182         la      $a1, player_count
183         jal     deal_card
184         la      $a0, dealer_hand
185         la      $a1, dealer_count
186         jal     deal_card
187
188         la      $t0, msg_hit_stand
189         sw      $t0, cur_msg
190         li      $a0, 1                  # dealer's 2nd card hidden
191         jal     redraw_table
192         jal     print_table_hidden
193
194         # natural blackjack check
195         la      $a0, player_hand
196         lw      $a1, player_count
197         jal     hand_value
198         li      $t0, 21
199         bne     $v0, $t0, player_turn
200         la      $a0, dealer_hand
201         lw      $a1, dealer_count
202         jal     hand_value
203         li      $t0, 21
204         beq     $v0, $t0, both_blackjack
205         la      $t0, msg_blackjack
206         sw      $t0, cur_msg
207         lw      $t1, wins
208         addi    $t1, $t1, 1
209         sw      $t1, wins
210         li      $a0, 0
211         jal     redraw_table
212         jal     print_dealer_full
213         print_str(c_blackjack)
214         j       round_end
215 both_blackjack:
216         la      $t0, msg_push
217         sw      $t0, cur_msg
218         lw      $t1, pushes
219         addi    $t1, $t1, 1
220         sw      $t1, pushes
221         li      $a0, 0
222         jal     redraw_table
223         jal     print_dealer_full
224         print_str(c_both_bj)
225         j       round_end
226
227 player_turn:
228         print_str(c_prompt)
229         la      $a0, input_buf
230         li      $a1, 64
231         jal     read_line
232         lb      $a0, input_buf
233         jal     to_upper
234         li      $t0, 'H'
235         beq     $v0, $t0, player_hits
236         li      $t0, 'S'
237         beq     $v0, $t0, dealer_turn
238         print_str(c_bad_hs)
239         j       player_turn
240
241 player_hits:
242         la      $a0, player_hand
243         la      $a1, player_count
244         jal     deal_card
245         move    $s2, $v0                # keep the card (print_str
        clobbers $v0)
246         print_str(c_you_draw)
247         move    $a0, $s2
248         jal     print_card
249         print_str(newline)
250         li      $a0, 1
251         jal     redraw_table
252         jal     print_player_line
```

```
253         la      $a0, player_hand
254         lw      $a1, player_count
255         jal     hand_value
256         bgt     $v0, 21, player_bust
257         li      $t0, 21
258         bne     $v0, $t0, player_turn
259         print_str(c_21_stand)
260         j       dealer_turn
261
262 player_bust:
263         la      $t0, msg_bust
264         sw      $t0, cur_msg
265         lw      $t1, losses
266         addi    $t1, $t1, 1
267         sw      $t1, losses
268         li      $a0, 0
269         jal     redraw_table
270         print_str(c_bust)
271         j       round_end
272
273 dealer_turn:
274         li      $a0, 0                  # reveal the hole card
275         jal     redraw_table
276         print_str(c_dealer_reveal)
277         la      $a0, dealer_hand
278         lw      $a1, dealer_count
279         jal     print_hand
280         print_str(newline)
281         li      $s3, 0                  # $s3 = 1 once the dealer
        has hit
282 dealer_loop:
283         la      $a0, dealer_hand
284         lw      $a1, dealer_count
285         jal     hand_value
286         bge     $v0, 17, dealer_done    # stands on 17 or more
287         la      $a0, dealer_hand
288         la      $a1, dealer_count
289         jal     deal_card
290         move    $s2, $v0                # keep the card (print_str
       clobbers $v0)
291         print_str(c_dealer_hits)
292         move    $a0, $s2
293         jal     print_card
294         print_str(newline)
295         li      $s3, 1
296         li      $a0, 0
297         jal     redraw_table
298         j       dealer_loop
299
300 dealer_done:
301         move    $s0, $v0                # dealer total
302         la      $a0, player_hand
303         lw      $a1, player_count
304         jal     hand_value
305         move    $s1, $v0                # player total
306         jal     print_dealer_full       # final dealer hand with t
      otal
307         bgt     $s0, 21, dealer_busted
308         bgt     $s1, $s0, player_wins
309         blt     $s1, $s0, dealer_wins
310         # push
311         la      $t0, msg_push
312         sw      $t0, cur_msg
313         lw      $t1, pushes
314         addi    $t1, $t1, 1
315         sw      $t1, pushes
316         print_str(c_push)
317         j       round_end_redraw
318 dealer_busted:
319         la      $t0, msg_dealer_bust
320         sw      $t0, cur_msg
321         lw      $t1, wins
322         addi    $t1, $t1, 1
323         sw      $t1, wins
324         print_str(c_dealer_bust)
325         j       round_end_redraw
326 player_wins:
327         la      $t0, msg_you_win
328         sw      $t0, cur_msg
329         lw      $t1, wins
330         addi    $t1, $t1, 1
331         sw      $t1, wins
332         print_str(c_you_win)
333         j       round_end_redraw
334 dealer_wins:
335         la      $t0, msg_dealer_win
336         sw      $t0, cur_msg
337         lw      $t1, losses
338         addi    $t1, $t1, 1
339         sw      $t1, losses
340         print_str(c_dealer_wins)
341
342 round_end_redraw:
343         li      $a0, 0
344         jal     redraw_table
345 round_end:
346         print_str(c_record1)
347         lw      $t0, wins
348         print_int($t0)
349         print_str(c_record2)
350         lw      $t0, losses
351         print_int($t0)
352         print_str(c_record3)
353         lw      $t0, pushes
354         print_int($t0)
355         print_str(c_record4)
356
357 ask_again:
358         print_str(c_again)
359         la      $a0, input_buf
360         li      $a1, 64
361         jal     read_line
362         lb      $a0, input_buf
363         jal     to_upper
364         li      $t0, 'Y'
365         beq     $v0, $t0, round_start
366         li      $t0, 'N'
367         beq     $v0, $t0, exit_game
368         print_str(c_bad_yn)
369         j       ask_again
370
371 exit_game:
372         print_str(c_bye)
373         li      $v0, 10
374         syscall
375
376 ################################################################
        ##############
377 #  =====================  DECK / HAND PROCEDURES  ==============
        =======
378 ################################################################
        ##############
379
380 # shuffle_deck: fills deck with 0..51 then Fisher-Yates shuffles i
        t
381 shuffle_deck:
382         la      $t0, deck
383         li      $t1, 0
384 sd_fill:
385         sb      $t1, 0($t0)
386         addi    $t0, $t0, 1
387         addi    $t1, $t1, 1
388         blt     $t1, 52, sd_fill
389         sw      $zero, deck_pos
390
391         li      $t2, 51                 # i = 51 down to 1
392 sd_loop:
393         li      $a0, 0                  # generator id
394         addi    $a1, $t2, 1             # upper bound (exclusive)
        = i + 1
395         li      $v0, 42
396         syscall                         # $a0 = j in [0, i]
397         la      $t0, deck
398         add     $t3, $t0, $t2           # &deck[i]
399         add     $t4, $t0, $a0           # &deck[j]
400         lb      $t5, 0($t3)
401         lb      $t6, 0($t4)
402         sb      $t6, 0($t3)
403         sb      $t5, 0($t4)             # swap
404         addi    $t2, $t2, -1
405         bgtz    $t2, sd_loop
406         jr      $ra
407
408 # deal_card(hand_addr, count_addr) -> $v0 = card dealt; appends to
         the hand.
409 # Reshuffles if the deck is somehow exhausted (defensive edge case
        ).
410 deal_card:
411         addi    $sp, $sp, -12
412         sw      $ra, 8($sp)
413         sw      $s0, 4($sp)
414         sw      $s1, 0($sp)
415         move    $s0, $a0
416         move    $s1, $a1
417         lw      $t0, deck_pos
418         blt     $t0, 52, dc_ok
419         jal     shuffle_deck
420         lw      $t0, deck_pos
421 dc_ok:
422         lb      $v0, deck($t0)
423         addi    $t0, $t0, 1
424         sw      $t0, deck_pos
425         lw      $t1, 0($s1)             # current count
426         bge     $t1, MAX_HAND, dc_done  # hand full: ignore (canno
        t happen in play)
427         add     $t2, $s0, $t1
428         sb      $v0, 0($t2)
429         addi    $t1, $t1, 1
430         sw      $t1, 0($s1)
431 dc_done:
432         lw      $ra, 8($sp)
433         lw      $s0, 4($sp)
```

```
        lw      $s1, 0($sp)
        addi    $sp, $sp, 12
        jr      $ra

# card_value(card) -> $v0 = 11 for an ace, 10 for 10/J/Q/K, else the pip value
card_value:
        li      $t0, 13
        div     $a0, $t0
        mfhi    $t1                     # rank
        bnez    $t1, cv_not_ace
        li      $v0, 11
        jr      $ra
cv_not_ace:
        addi    $v0, $t1, 1             # rank 1..8 -> 2..9, 9 -> 10
        ble     $v0, 10, cv_ret
        li      $v0, 10                 # J, Q, K
cv_ret:
        jr      $ra

# hand_value(hand_addr, count) -> $v0 = best total (aces drop from 11 to 1)
hand_value:
        addi    $sp, $sp, -20
        sw      $ra, 16($sp)
        sw      $s0, 12($sp)
        sw      $s1, 8($sp)
        sw      $s2, 4($sp)
        sw      $s3, 0($sp)
        move    $s0, $a0                # pointer
        move    $s1, $a1                # remaining
        li      $s2, 0                  # total
        li      $s3, 0                  # aces counted as 11
hv_loop:
        blez    $s1, hv_adjust
        lb      $a0, 0($s0)
        jal     card_value
        add     $s2, $s2, $v0
        li      $t0, 11
        bne     $v0, $t0, hv_next
        addi    $s3, $s3, 1
hv_next:
        addi    $s0, $s0, 1
        addi    $s1, $s1, -1
        j       hv_loop
hv_adjust:
        ble     $s2, 21, hv_done
        blez    $s3, hv_done
        addi    $s2, $s2, -10           # one ace: 11 -> 1
        addi    $s3, $s3, -1
        j       hv_adjust
hv_done:
        move    $v0, $s2
        lw      $ra, 16($sp)
        lw      $s0, 12($sp)
        lw      $s1, 8($sp)
        lw      $s2, 4($sp)
        lw      $s3, 0($sp)
        addi    $sp, $sp, 20
        jr      $ra

# to_upper(char) -> $v0 = upper-case version of an ASCII letter
to_upper:
        move    $v0, $a0
        blt     $a0, 'a', tu_ret
        bgt     $a0, 'z', tu_ret
        addi    $v0, $a0, -32
tu_ret:
        jr      $ra

################################################################################
#  ======================  CONSOLE OUTPUT  ======================
################################################################################

# print_card(card): prints e.g. "AS", "10H", "KD" followed by a space
print_card:
        addi    $sp, $sp, -8
        sw      $ra, 4($sp)
        sw      $s0, 0($sp)
        move    $s0, $a0
        li      $t0, 13
        div     $s0, $t0
        mfhi    $t1                     # rank
        mflo    $t2                     # suit
        bnez    $t1, pc_not_ace
        li      $t3, 'A'
        j       pc_one_char
pc_not_ace:
        li      $t4, 9
        bne     $t1, $t4, pc_not_ten
        print_str(rank_10)
        j       pc_suit
pc_not_ten:
        li      $t4, 10
        beq     $t1, $t4, pc_jack
        li      $t4, 11
        beq     $t1, $t4, pc_queen
        li      $t4, 12
        beq     $t1, $t4, pc_king
        addi    $t3, $t1, '1'           # rank 1..8 -> '2'..'9'
        j       pc_one_char
pc_jack:
        li      $t3, 'J'
        j       pc_one_char
pc_queen:
        li      $t3, 'Q'
        j       pc_one_char
pc_king:
        li      $t3, 'K'
pc_one_char:
        print_char($t3)
pc_suit:
        li      $t0, 13
        div     $s0, $t0
        mflo    $t2
        lb      $t3, suit_letters($t2)
        print_char($t3)
        print_str(space1)
        lw      $ra, 4($sp)
        lw      $s0, 0($sp)
        addi    $sp, $sp, 8
        jr      $ra

# print_hand(hand_addr, count): prints every card in the hand
print_hand:
        addi    $sp, $sp, -12
        sw      $ra, 8($sp)
        sw      $s0, 4($sp)
        sw      $s1, 0($sp)
        move    $s0, $a0
        move    $s1, $a1
ph_loop:
        blez    $s1, ph_done
        lb      $a0, 0($s0)
        jal     print_card
        addi    $s0, $s0, 1
        addi    $s1, $s1, -1
        j       ph_loop
ph_done:
        lw      $ra, 8($sp)
        lw      $s0, 4($sp)
        lw      $s1, 0($sp)
        addi    $sp, $sp, 12
        jr      $ra

# print_player_line: "Your hand    : 7H AD   (total 18)"
print_player_line:
        addi    $sp, $sp, -4
        sw      $ra, 0($sp)
        print_str(c_your_hand)
        la      $a0, player_hand
        lw      $a1, player_count
        jal     print_hand
        print_str(c_total)
        la      $a0, player_hand
        lw      $a1, player_count
        jal     hand_value
        print_int($v0)
        print_str(c_close)
        lw      $ra, 0($sp)
        addi    $sp, $sp, 4
        jr      $ra

# print_table_hidden: dealer's first card + [??], then the player line
print_table_hidden:
        addi    $sp, $sp, -4
        sw      $ra, 0($sp)
        print_str(c_dealer_shows)
        lb      $a0, dealer_hand
        jal     print_card
        print_str(c_hidden)
        print_str(newline)
        jal     print_player_line
        lw      $ra, 0($sp)
        addi    $sp, $sp, 4
        jr      $ra

# print_dealer_full: the whole dealer hand with its total
print_dealer_full:
        addi    $sp, $sp, -4
        sw      $ra, 0($sp)
        print_str(c_dealer_hand)
        la      $a0, dealer_hand
        lw      $a1, dealer_count
        jal     print_hand
        print_str(c_total)
        la      $a0, dealer_hand
```

```
        lw      $a1, dealer_count
        jal     hand_value
        print_int($v0)
        print_str(c_close)
        lw      $ra, 0($sp)
        addi    $sp, $sp, 4
        jr      $ra

#####################################################################
#  =====================  TABLE DRAWING  =====================
#####################################################################

# redraw_table(hide_dealer): repaints the whole display from the game state
redraw_table:
        addi    $sp, $sp, -8
        sw      $ra, 4($sp)
        sw      $s0, 0($sp)
        move    $s0, $a0                # hide flag

        li      $a0, COLOR_FELT
        jal     fill_screen

        li      $a0, 2
        li      $a1, 1
        la      $a2, dealer_lbl
        li      $a3, COLOR_TEXT
        jal     draw_text

        la      $a0, dealer_hand
        lw      $a1, dealer_count
        li      $a2, DEALER_Y
        move    $a3, $s0
        jal     draw_hand

        li      $a0, 2
        li      $a1, 21
        la      $a2, player_lbl
        li      $a3, COLOR_TEXT
        jal     draw_text

        la      $a0, player_hand
        lw      $a1, player_count
        li      $a2, PLAYER_Y
        li      $a3, 0
        jal     draw_hand

        move    $a0, $s0
        jal     draw_scores

        li      $a0, 2
        li      $a1, 48
        lw      $a2, cur_msg
        li      $a3, COLOR_GOLD
        jal     draw_text

        jal     draw_record

        lw      $ra, 4($sp)
        lw      $s0, 0($sp)
        addi    $sp, $sp, 8
        jr      $ra

# draw_hand(hand_addr, count, y, hide_second): draws a row of cards at x = 2 + 10*i
draw_hand:
        addi    $sp, $sp, -24
        sw      $ra, 20($sp)
        sw      $s0, 16($sp)
        sw      $s1, 12($sp)
        sw      $s2, 8($sp)
        sw      $s3, 4($sp)
        sw      $s4, 0($sp)
        move    $s0, $a0                # pointer
        move    $s1, $a1                # count
        move    $s2, $a2                # y
        move    $s3, $a3                # hide flag
        li      $s4, 0                  # index
dh_loop:
        bge     $s4, $s1, dh_done
        mul     $a0, $s4, CARD_PITCH
        addi    $a0, $a0, 2             # x
        move    $a1, $s2                # y
        add     $t0, $s0, $s4
        lb      $a2, 0($t0)             # card
        li      $a3, 1                  # face up
        beqz    $s3, dh_draw
        li      $t1, 1
        bne     $s4, $t1, dh_draw
        li      $a3, 0                  # second card face down
dh_draw:
        jal     draw_card
        addi    $s4, $s4, 1
        j       dh_loop
dh_done:
        lw      $ra, 20($sp)
        lw      $s0, 16($sp)
        lw      $s1, 12($sp)
        lw      $s2, 8($sp)
        lw      $s3, 4($sp)
        lw      $s4, 0($sp)
        addi    $sp, $sp, 24
        jr      $ra

# draw_card(x, y, card, face_up): 9 x 12 card with rank and suit letter
draw_card:
        addi    $sp, $sp, -24
        sw      $ra, 20($sp)
        sw      $s0, 16($sp)
        sw      $s1, 12($sp)
        sw      $s2, 8($sp)
        sw      $s3, 4($sp)
        sw      $s4, 0($sp)
        move    $s0, $a0                # x
        move    $s1, $a1                # y
        move    $s2, $a2                # card
        move    $s3, $a3                # face_up
        bnez    $s3, dcard_face

        # ---- face down: dark back with a light inner rectangle ----
        move    $a0, $s0
        move    $a1, $s1
        li      $a2, CARD_W
        li      $a3, CARD_H
        li      $t0, COLOR_BACK
        addi    $sp, $sp, -4
        sw      $t0, 0($sp)
        jal     draw_rect
        addi    $sp, $sp, 4
        addi    $a0, $s0, 2
        addi    $a1, $s1, 2
        li      $a2, 5
        li      $a3, 8
        li      $t0, COLOR_BACK2
        addi    $sp, $sp, -4
        sw      $t0, 0($sp)
        jal     draw_rect
        addi    $sp, $sp, 4
        addi    $a0, $s0, 3
        addi    $a1, $s1, 3
        li      $a2, 3
        li      $a3, 6
        li      $t0, COLOR_BACK
        addi    $sp, $sp, -4
        sw      $t0, 0($sp)
        jal     draw_rect
        addi    $sp, $sp, 4
        j       dcard_done

dcard_face:
        move    $a0, $s0
        move    $a1, $s1
        li      $a2, CARD_W
        li      $a3, CARD_H
        li      $t0, COLOR_CARD
        addi    $sp, $sp, -4
        sw      $t0, 0($sp)
        jal     draw_rect
        addi    $sp, $sp, 4

        # ink color from the suit: hearts(1)/diamonds(2) red, else black
        li      $t0, 13
        div     $s2, $t0
        mflo    $t1                     # suit
        mfhi    $t2                     # rank
        li      $s4, COLOR_BLACK
        li      $t3, 1
        beq     $t1, $t3, dcard_red
        li      $t3, 2
        bne     $t1, $t3, dcard_rank
dcard_red:
        li      $s4, COLOR_RED
dcard_rank:
        # rank glyph(s) at (x+1, y+1)
        li      $t3, 9
        bne     $t2, $t3, dcard_single
        addi    $a0, $s0, 1             # "10" needs two glyphs
        addi    $a1, $s1, 1
        la      $a2, rank_10
        move    $a3, $s4
        jal     draw_text
        j       dcard_suit
dcard_single:
        move    $a0, $t2
        jal     rank_char               # $v0 = 'A','2'..'9','J','Q','K'
        addi    $a0, $s0, 1
```

```
803         addi    $a1, $s1, 1
804         move    $a2, $v0
805         move    $a3, $s4
806         jal     draw_glyph
807 dcard_suit:
808         li      $t0, 13
809         div     $s2, $t0
810         mflo    $t1
811         lb      $a2, suit_letters($t1)
812         addi    $a0, $s0, 5
813         addi    $a1, $s1, 6
814         move    $a3, $s4
815         jal     draw_glyph
816 dcard_done:
817         lw      $ra, 20($sp)
818         lw      $s0, 16($sp)
819         lw      $s1, 12($sp)
820         lw      $s2, 8($sp)
821         lw      $s3, 4($sp)
822         lw      $s4, 0($sp)
823         addi    $sp, $sp, 24
824         jr      $ra
825
826 # rank_char(rank 0..12) -> $v0 = ASCII character for the rank (10
       handled by caller)
827 rank_char:
828         li      $v0, 'A'
829         beqz    $a0, rc_ret
830         li      $v0, 'J'
831         li      $t0, 10
832         beq     $a0, $t0, rc_ret
833         li      $v0, 'Q'
834         li      $t0, 11
835         beq     $a0, $t0, rc_ret
836         li      $v0, 'K'
837         li      $t0, 12
838         beq     $a0, $t0, rc_ret
839         addi    $v0, $a0, '1'           # 1..8 -> '2'..'9'
840 rc_ret:
841         jr      $ra
842
843 # draw_scores(hide_dealer): row 41 -> "YOU nn DLR nn" (or "DLR ?"
       while hidden)
844 draw_scores:
845         addi    $sp, $sp, -8
846         sw      $ra, 4($sp)
847         sw      $s0, 0($sp)
848         move    $s0, $a0
849         li      $a0, 2
850         li      $a1, 41
851         la      $a2, you_lbl
852         li      $a3, COLOR_TEXT
853         jal     draw_text
854         la      $a0, player_hand
855         lw      $a1, player_count
856         jal     hand_value
857         li      $a0, 16
858         li      $a1, 41
859         move    $a2, $v0
860         li      $a3, COLOR_GOLD
861         jal     draw_number
862         li      $a0, 30
863         li      $a1, 41
864         la      $a2, dlr_lbl
865         li      $a3, COLOR_TEXT
866         jal     draw_text
867         beqz    $s0, ds_show_dealer
868         li      $a0, 44
869         li      $a1, 41
870         la      $a2, unknown_lbl
871         li      $a3, COLOR_GOLD
872         jal     draw_text
873         j       ds_done
874 ds_show_dealer:
875         la      $a0, dealer_hand
876         lw      $a1, dealer_count
877         jal     hand_value
878         li      $a0, 44
879         li      $a1, 41
880         move    $a2, $v0
881         li      $a3, COLOR_GOLD
882         jal     draw_number
883 ds_done:
884         lw      $ra, 4($sp)
885         lw      $s0, 0($sp)
886         addi    $sp, $sp, 8
887         jr      $ra
888
889 # draw_record: row 55 -> "W n L n P n"
890 draw_record:
891         addi    $sp, $sp, -4
892         sw      $ra, 0($sp)
893         li      $a0, 2
894         li      $a1, 55
895         la      $a2, w_lbl
896         li      $a3, COLOR_TEXT
897         jal     draw_text
898         li      $a0, 8
899         li      $a1, 55
900         lw      $a2, wins
901         li      $a3, COLOR_GOLD
902         jal     draw_number
903         li      $a0, 22
904         li      $a1, 55
905         la      $a2, l_lbl
906         li      $a3, COLOR_TEXT
907         jal     draw_text
908         li      $a0, 28
909         li      $a1, 55
910         lw      $a2, losses
911         li      $a3, COLOR_GOLD
912         jal     draw_number
913         li      $a0, 42
914         li      $a1, 55
915         la      $a2, p_lbl
916         li      $a3, COLOR_TEXT
917         jal     draw_text
918         li      $a0, 48
919         li      $a1, 55
920         lw      $a2, pushes
921         li      $a3, COLOR_GOLD
922         jal     draw_number
923         lw      $ra, 0($sp)
924         addi    $sp, $sp, 4
925         jr      $ra
926
927 # draw_number(x, y, value, color): converts value to decimal text
       and draws it
928 draw_number:
929         addi    $sp, $sp, -4
930         sw      $ra, 0($sp)
931         move    $t9, $a2
932         move    $a2, $a3                # keep color in $a2 for a
       moment
933         move    $a3, $a1                # y
934         move    $t8, $a0                # x
935         move    $a0, $t9                # value
936         la      $a1, num_buf
937         move    $t7, $a2                # color
938         jal     int_to_str
939         move    $a0, $t8
940         move    $a1, $a3
941         la      $a2, num_buf
942         move    $a3, $t7
943         jal     draw_text
944         lw      $ra, 0($sp)
945         addi    $sp, $sp, 4
946         jr      $ra
947
948 # int_to_str(value >= 0, buffer): writes the decimal digits + NUL
949 int_to_str:
950         move    $t0, $a1
951         li      $t1, 10
952         li      $t2, 0                  # digit count
953         bnez    $a0, its_push
954         li      $t3, '0'
955         sb      $t3, 0($t0)
956         sb      $zero, 1($t0)
957         jr      $ra
958 its_push:                               # push digits (reversed) o
       n the stack
959         beqz    $a0, its_pop
960         div     $a0, $t1
961         mfhi    $t3
962         mflo    $a0
963         addi    $t3, $t3, '0'
964         addi    $sp, $sp, -4
965         sw      $t3, 0($sp)
966         addi    $t2, $t2, 1
967         j       its_push
968 its_pop:
969         beqz    $t2, its_end
970         lw      $t3, 0($sp)
971         addi    $sp, $sp, 4
972         sb      $t3, 0($t0)
973         addi    $t0, $t0, 1
974         addi    $t2, $t2, -1
975         j       its_pop
976 its_end:
977         sb      $zero, 0($t0)
978         jr      $ra
979
```

(Graphics library, draw_pixel through draw_text, and read_line omitted. Same code as tictactoe.asm lines 668-917.)

```
1181
```

hangman.asm

```
################################################################
#  hangman.asm  --  Hangman (custom game) for the MARS MIPS simulator
#
#  Module 4: MIPS Game Project  (Game 3 of 3 - my own custom game)
#  Author : Dylan Flores
#
#  HOW TO RUN (MARS 4.5)
#    1. Open this file in MARS and assemble it (F3).
#    2. Tools -> Bitmap Display.  Set:
#         Unit Width  = 4      Unit Height = 4
#         Display Width = 256  Display Height = 256
#         Base address for display = 0x10008000 ($gp)
#       Click "Connect to MIPS".
#    3. Run (F5).  Type single letters in the Run I/O console.
#
#  GAME
#    The program picks a random computer-architecture word from a word table.
#    The player guesses one letter at a time.  Correct letters are revealed in
#    the word row on the bitmap display; each wrong guess draws one more part
#    of the hanged figure (head, body, two arms, two legs = 6 lives).  Wrong
#    letters are listed in red.  Input is validated: exactly one character,
#    must be a letter (upper or lower case accepted), repeats are rejected
#    without costing a life.
#
#  DATA STRUCTURES
#    word_table  : array of pointers to NUL-terminated upper-case words
#    guessed[26] : byte flags, guessed[c - 'A'] = 1 once letter c is tried
#    wrong_buf   : the wrong letters in the order they were guessed
#
#  REGISTER / STACK CONVENTIONS USED
#    $a0-$a3 arguments, $v0-$v1 results, $t* caller-saved, $s* callee-saved
#    and saved on the stack by every procedure that uses them.
#    draw_rect takes a 5th argument (color) on the stack at 0($sp).
################################################################

        .eqv  DISPLAY_BASE  0x10008000
        .eqv  SCREEN_W      64
        .eqv  SCREEN_H      64
        .eqv  SCREEN_WORDS  4096

        .eqv  COLOR_BG      0x00181C28    # night blue background
        .eqv  COLOR_WOOD    0x00A0703A    # gallows
        .eqv  COLOR_ROPE    0x00D0B070
        .eqv  COLOR_BODY    0x00F0E0C0    # figure
        .eqv  COLOR_TEXT    0x00FFFFFF
        .eqv  COLOR_GOLD    0x00FFD040
        .eqv  COLOR_BAD     0x00FF5050
        .eqv  COLOR_GOOD    0x0050FF70

        .eqv  MAX_WRONG     6
        .eqv  WORD_COUNT    12
        .eqv  WORD_ROW_Y    44
        .eqv  MISS_ROW_Y    52

        .macro print_str(%label)
        la    $a0, %label
        li    $v0, 4
        syscall
        .end_macro

        .macro print_int(%reg)
        move  $a0, %reg
        li    $v0, 1
        syscall
        .end_macro

        .macro print_char(%reg)
        move  $a0, %reg
        li    $v0, 11
        syscall
        .end_macro

################################################################
        .data
################################################################
w0:             .asciiz "COMPUTER"
w1:             .asciiz "ASSEMBLY"
w2:             .asciiz "REGISTER"
w3:             .asciiz "PROCEDURE"
w4:             .asciiz "MEMORY"
w5:             .asciiz "BRANCH"
w6:             .asciiz "STACK"
w7:             .asciiz "SYSCALL"
w8:             .asciiz "PIPELINE"
w9:             .asciiz "COMPILER"
w10:            .asciiz "HARDWARE"
w11:            .asciiz "PROGRAM"
                .align 2
word_table:     .word  w0, w1, w2, w3, w4, w5, w6, w7, w8, w9, w10, w11

cur_word:       .word  0               # pointer to the secret word
wrong_count:    .word  0
games_won:      .word  0
games_lost:     .word  0
guessed:        .space 26              # 1 = letter already tried
wrong_buf:      .space 8               # wrong letters, NUL-terminated
                .align 2
input_buf:      .space 64
num_buf:        .space 12

# 3 x 5 pixel font (see tictactoe.asm for the index layout)
font:           .byte 7,5,5,5,7, 2,6,2,2,7, 7,1,7,4,7, 7,1,7,1,7, 5,5,7,1,1
                .byte 7,4,7,1,7, 7,4,7,5,7, 7,1,1,1,1, 7,5,7,5,7, 7,5,7,1,7
                .byte 2,5,7,5,5, 6,5,6,5,6, 7,4,4,4,7, 6,5,5,5,6, 7,4,7,4,7
                .byte 7,4,7,4,4, 7,4,5,5,7, 5,5,7,5,5, 7,2,2,2,7, 1,1,1,5,7
                .byte 5,5,6,5,5, 4,4,4,4,7, 5,7,7,5,5, 6,5,5,5,5, 7,5,5,5,7
                .byte 7,5,7,4,4, 7,5,5,7,1, 6,5,6,5,5, 7,4,7,1,7, 7,2,2,2,2
                .byte 5,5,5,5,7, 5,5,5,5,2, 5,5,7,7,5, 5,5,2,5,5, 5,5,2,2,2
                .byte 7,1,2,4,7, 0,0,0,0,0, 0,0,0,0,7, 7,1,3,0,2, 0,0,7,0,0
                .byte 0,2,0,2,0, 2,2,2,0,2
                .align 2

# ---- display strings ----
title_txt:      .asciiz "HANGMAN"
lives_txt:      .asciiz "LIVES"
miss_txt:       .asciiz "MISS"
win_txt:        .asciiz "YOU WIN"
lose_txt:       .asciiz "YOU LOSE"

# ---- console strings ----
welcome:        .asciiz "=== HANGMAN (MIPS) ===\nI am thinking of a computer-architecture word.\nGuess one letter at a time. You have 6 lives.\n\n"
c_word:         .asciiz "Word : "
c_lives:        .asciiz "   Lives: "
c_missed:       .asciiz "   Missed: "
c_prompt:       .asciiz "Guess a letter: "
c_err_len:      .asciiz "  ! Enter exactly one letter.\n"
c_err_alpha:    .asciiz "  ! That is not a letter (A-Z only).\n"
c_err_dup:      .asciiz "  ! You already tried that letter.\n"
c_hit1:         .asciiz "  Yes! '"
c_hit2:         .asciiz "' appears "
c_hit3:         .asciiz " time(s).\n"
c_miss1:        .asciiz "  No '"
c_miss2:        .asciiz "' in the word.\n"
c_win:          .asciiz "\n*** You got it! The word was "
c_lose:         .asciiz "\n*** Out of lives! The word was "
c_period:       .asciiz ". ***\n"
c_score1:       .asciiz "Score: "
c_score2:       .asciiz " won, "
c_score3:       .asciiz " lost.\n"
c_again:        .asciiz "Play again? (Y/N): "
c_bad_yn:       .asciiz "  ! Please answer Y or N.\n"
c_bye:          .asciiz "Thanks for playing!\n"
newline:        .asciiz "\n"
space1:         .asciiz " "

################################################################
        .text
        .globl main
################################################################
main:
        print_str(welcome)
        li      $v0, 30                # seed generator 0 from the clock
        syscall
        move    $a1, $a0
        li      $a0, 0
        li      $v0, 40
        syscall

```

```
new_game:
        jal     reset_game              # picks a word, clears guesses
        jal     draw_scene
        jal     print_state

turn:
        print_str(c_prompt)
        la      $a0, input_buf
        li      $a1, 64
        jal     read_line               # $v0 = length without newline
        li      $t0, 1
        bne     $v0, $t0, bad_length
        lb      $a0, input_buf
        jal     to_upper
        move    $s0, $v0                # $s0 = guessed letter (upper case)
        blt     $s0, 'A', bad_alpha
        bgt     $s0, 'Z', bad_alpha
        addi    $t0, $s0, -65           # index 0..25
        lb      $t1, guessed($t0)
        bnez    $t1, bad_dup
        li      $t1, 1
        sb      $t1, guessed($t0)       # remember the guess

        move    $a0, $s0
        jal     count_in_word           # $v0 = occurrences of the letter
        beqz    $v0, wrong_guess
        move    $s1, $v0
        print_str(c_hit1)
        print_char($s0)
        print_str(c_hit2)
        print_int($s1)
        print_str(c_hit3)
        j       after_guess

wrong_guess:
        lw      $t0, wrong_count
        la      $t1, wrong_buf
        add     $t1, $t1, $t0
        sb      $s0, 0($t1)             # append to the miss list
        sb      $zero, 1($t1)
        addi    $t0, $t0, 1
        sw      $t0, wrong_count
        print_str(c_miss1)
        print_char($s0)
        print_str(c_miss2)

after_guess:
        jal     draw_scene
        jal     print_state
        jal     word_solved             # $v0 = 1 if every letter is revealed
        bnez    $v0, game_won
        lw      $t0, wrong_count
        blt     $t0, MAX_WRONG, turn

        # ---- lost ----
        lw      $t0, games_lost
        addi    $t0, $t0, 1
        sw      $t0, games_lost
        li      $a0, 1
        jal     draw_word_row           # reveal the whole word (missing letters in red)
        li      $a0, 30
        li      $a1, 15
        la      $a2, lose_txt
        li      $a3, COLOR_BAD
        jal     draw_text
        print_str(c_lose)
        lw      $a0, cur_word
        li      $v0, 4
        syscall
        print_str(c_period)
        j       show_score

game_won:
        lw      $t0, games_won
        addi    $t0, $t0, 1
        sw      $t0, games_won
        li      $a0, 30
        li      $a1, 15
        la      $a2, win_txt
        li      $a3, COLOR_GOOD
        jal     draw_text
        print_str(c_win)
        lw      $a0, cur_word
        li      $v0, 4
        syscall
        print_str(c_period)

show_score:
        print_str(c_score1)
        lw      $t0, games_won
        print_int($t0)
        print_str(c_score2)
        lw      $t0, games_lost
        print_int($t0)
        print_str(c_score3)

ask_again:
        print_str(c_again)
        la      $a0, input_buf
        li      $a1, 64
        jal     read_line
        lb      $a0, input_buf
        jal     to_upper
        li      $t0, 'Y'
        beq     $v0, $t0, new_game
        li      $t0, 'N'
        beq     $v0, $t0, exit_game
        print_str(c_bad_yn)
        j       ask_again

bad_length:
        print_str(c_err_len)
        j       turn
bad_alpha:
        print_str(c_err_alpha)
        j       turn
bad_dup:
        print_str(c_err_dup)
        j       turn

exit_game:
        print_str(c_bye)
        li      $v0, 10
        syscall

################################################################
#  ======================  GAME STATE PROCEDURES  ================
################################################################

# reset_game: random word, clear guessed[] and the miss list
reset_game:
        li      $a0, 0
        li      $a1, WORD_COUNT
        li      $v0, 42
        syscall                         # $a0 = random index
        sll     $t0, $a0, 2
        lw      $t0, word_table($t0)
        sw      $t0, cur_word
        sw      $zero, wrong_count
        sb      $zero, wrong_buf
        la      $t0, guessed
        li      $t1, 26
rg_loop:
        sb      $zero, 0($t0)
        addi    $t0, $t0, 1
        addi    $t1, $t1, -1
        bnez    $t1, rg_loop
        jr      $ra

# count_in_word(letter) -> $v0 = how many times the letter occurs in the word
count_in_word:
        lw      $t0, cur_word
        li      $v0, 0
ciw_loop:
        lb      $t1, 0($t0)
        beqz    $t1, ciw_done
        bne     $t1, $a0, ciw_next
        addi    $v0, $v0, 1
ciw_next:
        addi    $t0, $t0, 1
        j       ciw_loop
ciw_done:
        jr      $ra

# word_solved -> $v0 = 1 if every letter of the word has been guessed
word_solved:
        lw      $t0, cur_word
ws_loop:
        lb      $t1, 0($t0)
        beqz    $t1, ws_yes
        addi    $t1, $t1, -65
        lb      $t2, guessed($t1)
        beqz    $t2, ws_no
        addi    $t0, $t0, 1
        j       ws_loop
ws_yes:
        li      $v0, 1
        jr      $ra
ws_no:
        li      $v0, 0
        jr      $ra
```

```
# to_upper(char) -> $v0
to_upper:
        move    $v0, $a0
        blt     $a0, 'a', tu_ret
        bgt     $a0, 'z', tu_ret
        addi    $v0, $a0, -32
tu_ret:
        jr      $ra

# print_state: "Word : _ E _ _   Lives: 4   Missed: XZ"
print_state:
        addi    $sp, $sp, -4
        sw      $ra, 0($sp)
        print_str(c_word)
        lw      $t5, cur_word
ps_loop:
        lb      $t6, 0($t5)
        beqz    $t6, ps_after_word
        addi    $t7, $t6, -65
        lb      $t7, guessed($t7)
        bnez    $t7, ps_show
        li      $t6, '_'
ps_show:
        print_char($t6)                 # macro only touches $a0/$v0
        print_str(space1)
        addi    $t5, $t5, 1
        j       ps_loop
ps_after_word:
        print_str(c_lives)
        lw      $t0, wrong_count
        li      $t1, MAX_WRONG
        sub     $t0, $t1, $t0
        print_int($t0)
        print_str(c_missed)
        la      $a0, wrong_buf
        li      $v0, 4
        syscall
        print_str(newline)
        lw      $ra, 0($sp)
        addi    $sp, $sp, 4
        jr      $ra

# ======================================================================
#  ======================  SCENE DRAWING  ======================
# ======================================================================

# draw_scene: repaints everything from the current game state
draw_scene:
        addi    $sp, $sp, -4
        sw      $ra, 0($sp)
        li      $a0, COLOR_BG
        jal     fill_screen
        li      $a0, 30
        li      $a1, 1
        la      $a2, title_txt
        li      $a3, COLOR_GOLD
        jal     draw_text
        jal     draw_lives
        jal     draw_gallows
        lw      $a0, wrong_count
        jal     draw_figure
        li      $a0, 0
        jal     draw_word_row
        jal     draw_miss_row
        lw      $ra, 0($sp)
        addi    $sp, $sp, 4
        jr      $ra

# draw_lives: "LIVES n" at (30, 8)
draw_lives:
        addi    $sp, $sp, -4
        sw      $ra, 0($sp)
        li      $a0, 30
        li      $a1, 8
        la      $a2, lives_txt
        li      $a3, COLOR_TEXT
        jal     draw_text
        lw      $t0, wrong_count
        li      $a2, MAX_WRONG
        sub     $a2, $a2, $t0
        li      $a0, 54
        li      $a1, 8
        li      $a3, COLOR_GOLD
        jal     draw_number
        lw      $ra, 0($sp)
        addi    $sp, $sp, 4
        jr      $ra

# draw_gallows: base, pole, beam and rope
draw_gallows:
        addi    $sp, $sp, -4
        sw      $ra, 0($sp)
        li      $a0, 2                  # base  (22 x 1) at y = 38
        li      $a1, 38
        li      $a2, 22
        li      $a3, 1
        li      $t0, COLOR_WOOD
        addi    $sp, $sp, -4
        sw      $t0, 0($sp)
        jal     draw_rect
        addi    $sp, $sp, 4
        li      $a0, 5                  # pole  (1 x 34) at x = 5
        li      $a1, 4
        li      $a2, 1
        li      $a3, 34
        li      $t0, COLOR_WOOD
        addi    $sp, $sp, -4
        sw      $t0, 0($sp)
        jal     draw_rect
        addi    $sp, $sp, 4
        li      $a0, 5                  # beam  (13 x 1) at y = 4
        li      $a1, 4
        li      $a2, 13
        li      $a3, 1
        li      $t0, COLOR_WOOD
        addi    $sp, $sp, -4
        sw      $t0, 0($sp)
        jal     draw_rect
        addi    $sp, $sp, 4
        li      $a0, 17                 # rope  (1 x 3) at x = 17
        li      $a1, 5
        li      $a2, 1
        li      $a3, 3
        li      $t0, COLOR_ROPE
        addi    $sp, $sp, -4
        sw      $t0, 0($sp)
        jal     draw_rect
        addi    $sp, $sp, 4
        lw      $ra, 0($sp)
        addi    $sp, $sp, 4
        jr      $ra

# draw_figure(parts): draws the first `parts` body parts (0..6)
draw_figure:
        addi    $sp, $sp, -8
        sw      $ra, 4($sp)
        sw      $s0, 0($sp)
        move    $s0, $a0
        blez    $s0, df_done
        # 1: head - 5x5 ring centred on x = 17
        li      $a0, 15
        li      $a1, 8
        li      $a2, 5
        li      $a3, 5
        li      $t0, COLOR_BODY
        addi    $sp, $sp, -4
        sw      $t0, 0($sp)
        jal     draw_rect
        addi    $sp, $sp, 4
        li      $a0, 16
        li      $a1, 9
        li      $a2, 3
        li      $a3, 3
        li      $t0, COLOR_BG
        addi    $sp, $sp, -4
        sw      $t0, 0($sp)
        jal     draw_rect
        addi    $sp, $sp, 4
        li      $a0, 15
        li      $a1, 8
        li      $a2, COLOR_BG
        jal     draw_pixel
        li      $a0, 19
        li      $a1, 8
        li      $a2, COLOR_BG
        jal     draw_pixel
        li      $a0, 15
        li      $a1, 12
        li      $a2, COLOR_BG
        jal     draw_pixel
        li      $a0, 19
        li      $a1, 12
        li      $a2, COLOR_BG
        jal     draw_pixel
        li      $t0, 1
        beq     $s0, $t0, df_done
        # 2: body - 1 x 9 line
        li      $a0, 17
        li      $a1, 13
        li      $a2, 1
        li      $a3, 9
        li      $t0, COLOR_BODY
        addi    $sp, $sp, -4
        sw      $t0, 0($sp)
        jal     draw_rect
        addi    $sp, $sp, 4
        li      $t0, 2
        beq     $s0, $t0, df_done
```

```
        # 3: left arm - diagonal down-left from (16,14)
        li      $a0, 16
        li      $a1, 14
        li      $a2, -1
        jal     draw_diag
        li      $t0, 3
        beq     $s0, $t0, df_done
        # 4: right arm - diagonal down-right from (18,14)
        li      $a0, 18
        li      $a1, 14
        li      $a2, 1
        jal     draw_diag
        li      $t0, 4
        beq     $s0, $t0, df_done
        # 5: left leg from (16,22)
        li      $a0, 16
        li      $a1, 22
        li      $a2, -1
        jal     draw_diag
        li      $t0, 5
        beq     $s0, $t0, df_done
        # 6: right leg from (18,22)
        li      $a0, 18
        li      $a1, 22
        li      $a2, 1
        jal     draw_diag
df_done:
        lw      $ra, 4($sp)
        lw      $s0, 0($sp)
        addi    $sp, $sp, 8
        jr      $ra

# draw_diag(x, y, dx): 4 pixels stepping (dx, +1) each time, in the body color
draw_diag:
        addi    $sp, $sp, -20
        sw      $ra, 16($sp)
        sw      $s0, 12($sp)
        sw      $s1, 8($sp)
        sw      $s2, 4($sp)
        sw      $s3, 0($sp)
        move    $s0, $a0
        move    $s1, $a1
        move    $s2, $a2
        li      $s3, 4
dd_loop:
        move    $a0, $s0
        move    $a1, $s1
        li      $a2, COLOR_BODY
        jal     draw_pixel
        add     $s0, $s0, $s2
        addi    $s1, $s1, 1
        addi    $s3, $s3, -1
        bnez    $s3, dd_loop
        lw      $ra, 16($sp)
        lw      $s0, 12($sp)
        lw      $s1, 8($sp)
        lw      $s2, 4($sp)
        lw      $s3, 0($sp)
        addi    $sp, $sp, 20
        jr      $ra

# draw_word_row(reveal_all): letters guessed -> white, unknown -> gold '_'
# (or red letter when reveal_all = 1 after a loss)
draw_word_row:
        addi    $sp, $sp, -16
        sw      $ra, 12($sp)
        sw      $s0, 8($sp)
        sw      $s1, 4($sp)
        sw      $s2, 0($sp)
        move    $s2, $a0                # reveal flag
        lw      $s0, cur_word
        li      $s1, 2                  # x position
dwr_loop:
        lb      $a2, 0($s0)
        beqz    $a2, dwr_done
        addi    $t0, $a2, -65
        lb      $t0, guessed($t0)
        li      $a3, COLOR_TEXT
        bnez    $t0, dwr_draw
        li      $a3, COLOR_BAD
        bnez    $s2, dwr_draw           # reveal missing letters in red
        li      $a2, '_'
        li      $a3, COLOR_GOLD
dwr_draw:
        move    $a0, $s1
        li      $a1, WORD_ROW_Y
        jal     draw_glyph
        addi    $s1, $s1, 4
        addi    $s0, $s0, 1
        j       dwr_loop
dwr_done:
        lw      $ra, 12($sp)
        lw      $s0, 8($sp)
        lw      $s1, 4($sp)
        lw      $s2, 0($sp)
        addi    $sp, $sp, 16
        jr      $ra

# draw_miss_row: "MISS" label followed by the wrong letters in red
draw_miss_row:
        addi    $sp, $sp, -4
        sw      $ra, 0($sp)
        li      $a0, 2
        li      $a1, MISS_ROW_Y
        la      $a2, miss_txt
        li      $a3, COLOR_TEXT
        jal     draw_text
        li      $a0, 22
        li      $a1, MISS_ROW_Y
        la      $a2, wrong_buf
        li      $a3, COLOR_BAD
        jal     draw_text
        lw      $ra, 0($sp)
        addi    $sp, $sp, 4
        jr      $ra

# draw_number(x, y, value, color)
draw_number:
        addi    $sp, $sp, -4
        sw      $ra, 0($sp)
        move    $t8, $a0                # x
        move    $t7, $a1                # y
        move    $t6, $a3                # color
        move    $a0, $a2
        la      $a1, num_buf
        jal     int_to_str
        move    $a0, $t8
        move    $a1, $t7
        la      $a2, num_buf
        move    $a3, $t6
        jal     draw_text
        lw      $ra, 0($sp)
        addi    $sp, $sp, 4
        jr      $ra

# int_to_str(value >= 0, buffer)
int_to_str:
        move    $t0, $a1
        li      $t1, 10
        li      $t2, 0
        bnez    $a0, its_push
        li      $t3, '0'
        sb      $t3, 0($t0)
        sb      $zero, 1($t0)
        jr      $ra
its_push:
        beqz    $a0, its_pop
        div     $a0, $t1
        mfhi    $t3
        mflo    $a0
        addi    $t3, $t3, '0'
        addi    $sp, $sp, -4
        sw      $t3, 0($sp)
        addi    $t2, $t2, 1
        j       its_push
its_pop:
        beqz    $t2, its_end
        lw      $t3, 0($sp)
        addi    $sp, $sp, 4
        sb      $t3, 0($t0)
        addi    $t0, $t0, 1
        addi    $t2, $t2, -1
        j       its_pop
its_end:
        sb      $zero, 0($t0)
        jr      $ra
```

(Graphics library, draw_pixel through draw_text, and read_line omitted. Same code as tictactoe.asm lines 668-917.)

```
897
```

Sources and acknowledgments

- ZyBooks Chapter 4 for procedures, the stack, arrays and branching.
- The MARS help menu for the syscall numbers (4, 5, 8, 10, 11, 30, 40, 42) and the Bitmap Display settings. MARS 4.5 is by Pete Sanderson and Ken Vollmar (dpetersanderson.github.io).
- Fisher-Yates shuffle and the dealer's hit on 16 stand on 17 rule are the standard way to do those, no code was copied.
- For testing I ran the programs in MARS from the command line with scripted input and used Python scripts to type the answers, dump the display memory to PNG files and double check the blackjack math. Java was OpenJDK 21.
- I used Claude, an AI assistant, to help write and debug the MIPS and Java code, the test scripts and this write up. I went through the code, ran all the tests, and I am responsible for the submission.
- I did not work with other students on this.